

ripoint: An Interior Point Optimizer in Rust with Implicit Slacks and Robust Recovery

John R. Kitchen

May 18, 2026

Note: This manuscript is generated and updated by Claude Code, based on the ripoint codebase and its development history. The content reflects the design decisions, algorithmic innovations, and benchmarking results that emerged during the implementation process. The manuscript is structured to provide a comprehensive overview of ripoint’s architecture, key contributions, and performance relative to Ipopt. The author (John R. Kitchen) provided high-level guidance and review but did not write any of the code or manuscript text directly (besides this note).

See supporting-information.org for links to the full source code, test suite, and benchmarking infrastructure.

Contents

1	Abstract	4
2	Introduction	5
2.1	Nonlinear Programming and Interior Point Methods	5
2.2	The Ipopt Algorithm	5
2.3	Motivation for ripoint	6
2.4	Contributions	6
2.5	Notation and Conventions	8
3	Implicit Slack Formulation	8
3.1	Standard Approach: Explicit Slacks	8
3.2	Our Approach: Implicit Slacks	8
3.3	System Size Reduction	9
3.4	Handling Infeasible Iterates	9
3.5	Condensed KKT via Schur Complement	9
4	Convergence Test Aligned with Ipopt	10
4.1	Iterative Bound Multipliers	10
4.2	Two-Gate Test on Iterative Multipliers	10

5	Two-Phase Restoration with Multi-Attempt Recovery	11
5.1	Background	11
5.2	Soft Restoration	11
5.3	Phase 1: Gauss-Newton Feasibility Solver	11
5.4	Phase 2: NLP Restoration	12
5.5	Multi-Attempt Recovery	12
5.6	Restoration Convergence: Three-Gate Test	13
5.7	Post-Restoration State Handoff	14
6	Preprocessing	15
6.1	Fixed Variable Elimination	15
6.2	Redundant Constraint Detection (legacy, default-on)	15
6.3	Bound Tightening from Linear Constraints (legacy, default-on)	16
7	NLP Scaling	16
8	Near-Linear Constraint Detection	16
9	Multi-Solver Fallback Architecture	17
9.1	L-BFGS for Unconstrained Problems	17
9.2	Augmented Lagrangian for Constrained Problems	17
9.3	SQP Fallback for Constrained Problems	18
9.4	Explicit Slack Reformulation	18
9.5	Best-Point Tracking for Cycling Detection	18
10	Adaptive Barrier Parameter	19
10.1	Oracles	19
10.2	Free and Fixed Modes	19
10.3	Additional Design Choices	20
11	Filter Line Search	20
11.1	Switching Condition	20
11.2	Second-Order Correction	21
11.3	Objective Safeguard	21
11.4	Filter Augmentation and Acceptance Convention	21
12	Linear Algebra	21
12.1	Bunch-Kaufman Factorization	21
12.2	Sparse LDL^T Factorization	22
12.3	Inertia Correction	22
12.4	Iterative Refinement	23
12.5	KKT-Aware Factorization for Indefinite Systems	23
	12.5.1 feral (default v0.8 backend)	24
	12.5.2 rmumps (legacy v0.7 backend)	24
12.6	Nested Dissection Ordering	25
12.7	Iterative Solver: Preconditioned MINRES	26

12.8	Hybrid Direct-Iterative Solver	26
12.9	Configurable Stall Detection	27
13	Implementation	27
13.1	Language and Dependencies	27
13.2	Problem Interface	28
13.3	Performance Characteristics	28
13.4	AMPL NL Interface and Pyomo Integration	29
13.5	C API	30
13.5.1	Design	30
13.5.2	Usage Example	31
13.5.3	Memory Safety and Thread Safety	31
13.6	GAMS Solver Link	31
13.7	Julia/JuMP Interface	32
14	Computational Results	33
14.1	Hock-Schittkowski Test Suite	33
14.1.1	Solve Rates	33
14.1.2	Solution Quality	33
14.1.3	Speed	33
14.1.4	Iteration Counts	34
14.2	CUTEst Benchmark Suite	34
14.2.1	Overall Solve Rates	34
14.2.2	Solution Quality	34
14.2.3	Speed Comparison	35
14.2.4	Attribution of ripopt-only Solves	35
14.3	Domain-Specific Benchmarks	36
14.3.1	Electrolyte Speciation	36
14.3.2	AC Optimal Power Flow (Electrical Grid)	36
14.3.3	Parameter Estimation	37
14.3.4	Gas Pipeline Networks	37
14.3.5	Water Distribution Networks	37
14.4	Failure Analysis	37
14.4.1	Common Failure Patterns	37
14.4.2	Comparison of Failure Modes	38
14.5	Speed vs. Accuracy Trade-off	39
15	Parametric Sensitivity Analysis	39
15.1	Implementation	40
15.2	Example: HS071 with Parametric Constraint Bound	40
16	Opportunities for Improvement	40
16.1	Restoration-Phase Robustness	41
16.2	Infeasibility Detection and Recovery	41
16.3	Adaptive Barrier Parameter Tuning	41

16.4 Multiplier Initialization	41
16.5 Performance Optimization for Large Problems	42
16.6 Quasi-Newton as Primary Solver	42
16.7 Scaling	42
16.8 Summary of Estimated Impact	43
17 Related Work	43
18 Conclusions	44
19 References	45

1 Abstract

We present `ript`, a nonlinear interior point optimizer implemented in Rust. `ript` originated as a translation of the `Ipt` algorithm but has diverged in several architectural and algorithmic respects. The core design decision—eliminating explicit slack variables for inequality constraints in favor of an implicit slack formulation—produces smaller Karush-Kuhn-Tucker (KKT) systems and a simplified problem interface. This formulation relies on the same `Ipt`-aligned two-gate convergence test (scaled and unscaled) on the iterative bound multipliers, which v0.7.0 showed to be sufficient once the barrier subproblem has been driven to a small μ . We introduce a two-phase restoration architecture that uses a fast Gauss-Newton feasibility solver before falling back to a full NLP restoration subproblem, combined with a multi-attempt recovery strategy that explores different barrier landscapes through systematic perturbation. A multi-solver fallback architecture—including L-BFGS for unconstrained problems, augmented Lagrangian for constrained problems, and explicit slack reformulation—provides additional robustness when the primary IPM fails. `ript` also provides sIPOPT-style parametric sensitivity analysis, computing how the optimal solution changes under parameter perturbations via a single KKT backsolve at near-zero cost. On the Hock-Schittkowski test suite (historical, retired from the v0.8 harness), `ript` solved 118 of 120 problems (98.3%), two more than `Ipt`’s 116 (96.7%), with a $20.8\times$ geometric mean speedup on commonly-solved problems. On the CUTEst benchmark suite of 727 problems at v0.8.1, `ript` solves 551 (75.8%) to strict `Optimal` versus `Ipt`’s 556 (76.5%)—a 5-problem gap in `Ipt`’s favor on strict convergence, narrowed (and reversed) on robustness and speed. The geometric mean wall-clock speedup is $7.9\times$ (median $10.6\times$) on the 529 commonly-Optimal problems. `ript` solves 22 CUTEst problems that `Ipt` cannot, and `Ipt` solves 27 that `ript` cannot; of `ript`’s 22 exclusive wins, 12 are nonlinear-equation (NE) systems where CUTEst’s `Ipt` wrapper rejects with degrees-of-freedom error `IptStatus(-10)` and `ript`’s implicit-slack IPM accepts them as standard equality-constrained NLPs. On domain-specific benchmarks, `ript` solves all 13 electrolyte speciation problems (vs. 12/13 for `Ipt`) and all 4 AC optimal power flow cases (`Ipt`: 4/4). The v0.8 release line moves the default sparse linear solver out of the `ript` tree and into `feral`,¹ an independently developed and independently benchmarked pure-Rust multifrontal LDL^T solver. The previous backend (`rmumps`, also pure Rust) was an embedded port that lived inside the `ript` repository, which made it structurally awkward to attribute failures cleanly between the

IPM and the linear solver; feral resolves that entanglement by carrying its own test corpus and reference solvers. We provide a detailed analysis of where each solver excels, the failure modes that remain, and opportunities for improvement.

2 Introduction

2.1 Nonlinear Programming and Interior Point Methods

We consider the general nonlinear programming (NLP) problem:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_L \leq g(x) \leq g_U \\ & x_L \leq x \leq x_U \end{aligned} \tag{1}$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the objective, $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ are the constraints, and x_L, x_U, g_L, g_U are bounds (with $\pm\infty$ permitted). Interior point methods^{2,3} replace the bound constraints with logarithmic barrier terms and solve a sequence of barrier subproblems parameterized by $\mu > 0$:

$$\min_x f(x) - \mu \sum_i \ln(x_i - x_{L,i}) - \mu \sum_i \ln(x_{U,i} - x_i) \quad \text{s.t.} \quad g_L \leq g(x) \leq g_U \tag{2}$$

As $\mu \rightarrow 0$, the barrier subproblem solutions converge to a solution of the original NLP. The first-order optimality conditions of the barrier problem yield the primal-dual system, which is solved by Newton’s method at each iteration.

2.2 The Ipopt Algorithm

Ipopt (Interior Point OPTimizer)⁴ is the most widely used open-source solver for large-scale nonlinear programming. It combines a primal-dual interior point method with a filter line search⁵ and an adaptive barrier parameter update strategy.⁶ Key design decisions in Ipopt include: (i) introduction of explicit slack variables for every inequality constraint, (ii) a quality-function-based barrier parameter oracle with golden-section search, (iii) a single NLP restoration phase for feasibility recovery, and (iv) reliance on external sparse symmetric indefinite linear solvers (MA27, MA57, MUMPS, or Pardiso).

Ipopt has proven highly effective across a wide range of applications, from chemical process optimization to robotics trajectory planning. However, its design carries computational overhead from explicit slack variables, complex barrier parameter selection, and the initialization and interop costs of Fortran-based linear algebra libraries. These overheads are particularly significant for small-to-medium problems where the per-iteration cost is dominated by setup rather than factorization.

2.3 Motivation for ripopt

This project began as an attempt to translate the Ipopt algorithm into Rust, motivated by these goals:

1. **Explore the limits of agentic code engineering** Claude Code was used exclusively to generate all the code in ripopt, including the core IPM engine, linear algebra routines, and the entire test suite and benchmarking infrastructure. This project serves as a case study in how far we can push agentic code generation for complex numerical algorithms.
2. **Eliminate external dependencies.** Ipopt requires Fortran linear algebra libraries (MUMPS, MA27/57, or Pardiso) that complicate installation, cross-compilation, and embedding in modern software systems. A pure Rust implementation with built-in linear solvers removes this friction. v0.8.0 completes this transition: the default build pulls only the pure-Rust `feral` multifrontal LDL^T solver¹ (with `rmumps` retained as an opt-in workspace crate), and the resulting library has no C/C++/Fortran dependencies.
3. **Reduce per-iteration overhead.** Rust’s zero-cost abstractions, stack allocation, and absence of garbage collection enable lower per-iteration times, especially for small problems where C/Fortran interop and dynamic memory allocation dominate.

We almost immediately learned a direct, line by line translation did not occur through agentic engineering. Instead, the development process was highly iterative and exploratory, with many design decisions revisited multiple times as we gained a deeper understanding of the algorithm and its failure modes. This led to several significant divergences from the original Ipopt design, introducing several algorithmic innovations that ultimately allowed it to solve more benchmark problems than the reference implementation. These were not all intentional—some were serendipitous discoveries that emerged from the process of debugging and improving the implementation—but they represent meaningful contributions to the state of the art in interior point optimization. Ultimately, we kept these innovations because they improved robustness and performance, even though they made the code less of a direct translation of Ipopt.

2.4 Contributions

The contributions of this paper are:

1. An implicit slack formulation for inequality constraints that reduces the KKT system dimension from $(n + m + n_{\text{ineq}})$ to $(n + m)$ and simplifies the algorithmic interface (Section [Implicit Slack Formulation](#)).
2. An Ipopt-aligned two-gate convergence test on the iterative bound multipliers, validated at v0.7.0 across HS, CUTEst, and domain benchmarks (Section [Convergence Test Aligned with Ipopt](#)).

3. A two-phase restoration architecture combining a fast Gauss-Newton feasibility solver with a full NLP restoration subproblem, and a multi-attempt recovery strategy that systematically perturbs the barrier landscape (Section [Two-Phase Restoration with Multi-Attempt Recovery](#)).
4. Extensive benchmarking on 727 CUTEst problems,⁷ with analysis of speed, accuracy, and failure modes relative to Ipopt (Section [Computational Results](#)). An earlier (v0.7) Hock-Schittkowski harness has been retired and is reported here only as historical context.
5. A multi-solver fallback architecture that applies L-BFGS, augmented Lagrangian, SQP, and explicit slack reformulation when the primary interior point method fails, significantly improving robustness (Section [Multi-Solver Fallback Architecture](#)).
6. Problem preprocessing that eliminates fixed variables, detects redundant constraints, and tightens variable bounds from verified linear constraints before solving (Section [Preprocessing](#)).
7. Near-linear constraint detection that identifies linear constraints by comparing Jacobian values at multiple points, enabling the solver to skip their second-order Hessian contributions (Section [Near-Linear Constraint Detection](#)).
8. A built-in AMPL NL file interface with automatic differentiation, enabling drop-in integration with Pyomo⁸ via a thin solver plugin (Section [AMPL NL Interface and Pyomo Integration](#)).
9. A C API that mirrors the Ipopt C interface, enabling direct linking from C, C++, Python (via `ctypes/cffi`), and other languages without subprocess overhead (Section [C API](#)).
10. A GAMS solver link that bridges GAMS’s GMO (GAMS Modeling Object) API to ripopt’s C API, enabling `=option nlp = ripopt;=` in GAMS models with automatic Hessian support, option file integration, and status mapping (Section [GAMS Solver Link](#)).
11. An iterative KKT solver based on preconditioned MINRES⁹ with an incomplete LDL^T preconditioner,¹⁰ and a hybrid direct-iterative solver that automatically switches between direct and iterative modes based on factorization cost and convergence behavior (Section [Iterative Solver: Preconditioned MINRES](#)). Under the legacy `rmumps` backend this is a fully-fledged Krylov path; under the v0.8 default `feral` backend it currently reduces to direct factorization with extra iterative refinement, with a true incomplete- LDL^T + MINRES path deferred to v0.9.
12. A decoupled default linear-algebra stack: v0.8.0 ships with `feral`¹ — a pure-Rust multifrontal LDL^T solver developed in a separate repository, with its own benchmark corpus and reference-solver test harness — as the default sparse backend. The v0.7 backend (`rmumps`) was also pure Rust, but lived inside the ripopt repository as a workspace member; failure attribution between the IPM and the linear solver was

structurally awkward because the linear solver had no test surface independent of ripopt’s CUTEst score. Moving the factorization to a separately benchmarked sister project resolves that entanglement and makes the build remain entirely Rust without an FFI step (Section [Linear Algebra](#)).

2.5 Notation and Conventions

We denote the number of variables by n and the number of constraints by m . The Jacobian of g is $J \in \mathbb{R}^{m \times n}$. Following the Ipopt convention, the Lagrangian is $\mathcal{L}(x, y) = f(x) + y^T g(x)$, where $y \in \mathbb{R}^m$ are the constraint multipliers. Bound multipliers $z_L, z_U \in \mathbb{R}^n$ are associated with the variable bounds $x \geq x_L$ and $x \leq x_U$ respectively. The barrier parameter is $\mu > 0$.

3 Implicit Slack Formulation

3.1 Standard Approach: Explicit Slacks

Ipopt converts inequality constraints $g_L \leq g(x) \leq g_U$ by introducing explicit slack variables s :

$$g(x) - s = 0, \quad g_L \leq s \leq g_U \quad (3)$$

The barrier function then includes terms $-\mu \ln(s_i - g_{L,i})$ and $-\mu \ln(g_{U,i} - s_i)$, and the KKT system has dimension $(n + n_{\text{ineq}} + m)$ —primal variables, slack variables, and constraint multipliers. This requires tracking s , its associated multipliers z_s , slack fraction-to-boundary checks, and careful slack initialization.⁴

3.2 Our Approach: Implicit Slacks

We treat the constraint slacks $s_L = g(x) - g_L$ and $s_U = g_U - g(x)$ as implicit functions of x and condense their barrier contributions directly into the KKT matrix during assembly. For each inequality constraint i with slack $s_{L,i} = g_i(x) - g_{L,i} > 0$ (feasible), we compute a barrier stiffness:

$$\Sigma_{s,i} = \frac{z_{s_L,i}}{s_{L,i}} + \frac{z_{s_U,i}}{s_{U,i}} \quad (4)$$

where the implicit slack multipliers are derived from the Lagrange multiplier y_i :

$$z_{s_L,i} = \begin{cases} -y_i & \text{if } y_i < 0 \\ \mu/\tilde{s}_{L,i} & \text{otherwise} \end{cases} \quad (5)$$

and $\tilde{s}_{L,i} = \max(s_{L,i}, \mu, 10^{-10})$ is a safeguarded slack. This derivation follows from the complementarity conditions of the barrier problem: at a barrier-optimal point for a lower-bounded constraint, the multiplier y_i and the slack multiplier $z_{s_L,i}$ satisfy $y_i = -z_{s_L,i}$ (with appropriate sign conventions).

The resulting KKT system is:

$$\begin{bmatrix} H + \Sigma_x + \delta_w I & J^T \\ J & -\Sigma_s^{-1} - \delta_c I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} r_d \\ r_p \end{bmatrix} \quad (6)$$

where $H = \nabla_{xx}^2 \mathcal{L}$ is the Lagrangian Hessian, Σ_x contains the barrier contributions from variable bounds, Σ_s^{-1} is diagonal with entries $1/\Sigma_{s,i}$ for inequality constraints and 0 for equalities, and δ_w, δ_c are regularization parameters from inertia correction (Section [Inertia Correction](#)).

3.3 System Size Reduction

The implicit slack formulation reduces the KKT system size:

System	Dimension	Factorization Cost
Ipopt (explicit slacks)	$n + n_{\text{ineq}} + m$	$O((n + n_{\text{ineq}} + m)^3)$
riopt (implicit slacks)	$n + m$	$O((n + m)^3)$
riopt (condensed, $m \geq 2n$)	n	$O(n^2 m + n^3)$

For a problem with $n = 100$ variables and $m = 200$ inequality constraints, Ipopt factors a 500×500 system; ripopt factors a 300×300 system, or 100×100 with the Schur complement condensation.

3.4 Handling Infeasible Iterates

When an inequality constraint is violated ($s_{L,i} < 0$), no meaningful barrier term exists. We handle this with a branch: infeasible constraints receive a pure feasibility-driving right-hand side $-(g_i - g_{L,i})$ without barrier contribution, while feasible constraints use the full barrier formulation with safeguarded slack floors. This branching logic is more complex than Ipopt's uniform treatment of slacks (which are always positive by construction) but avoids the need for explicit slack variables entirely.

3.5 Condensed KKT via Schur Complement

When $m \geq 2n$, we further reduce the system by Schur complement elimination of Δy :

$$S \Delta x = r_d + J^T (-D_c)^{-1} r_p \quad (7)$$

where $S = H + \Sigma_x + \delta_w I + J^T (-D_c)^{-1} J$ is the Schur complement and D_c is the $(2, 2)$ block diagonal. The multiplier update is recovered as $\Delta y = (-D_c)^{-1} (J \Delta x - r_p)$. For equality constraints where the $(2, 2)$ entry would be zero, we use a stiff-spring penalty of -10^{-16} to avoid dropping equalities from the Schur complement. This condensation is particularly effective for problems with many more constraints than variables, such as data fitting and optimal control discretizations.

4 Convergence Test Aligned with Ipopt

4.1 Iterative Bound Multipliers

Because ripopt does not carry explicit slack variables for inequality constraints, earlier releases (through v0.6.x) feared that the iterative bound multipliers z_L, z_U would be too unreliable for a strict convergence check: Newton updates safeguarded by κ_σ ⁴ clamp $z_i \cdot s_i \in [\mu/\kappa_\sigma, \kappa_\sigma\mu]$ with $\kappa_\sigma = 10^{10}$, and at degenerate points the multiplier split between y and z is not unique. To guard against those pathologies, v0.6.x computed auxiliary stationarity-based multipliers z_{opt} directly from $\nabla f + J^T y - z_L + z_U = 0$, gated them against μ , and used them in the dual-infeasibility term of a scaled convergence check.

Experience with the v0.7.0 test suite (HS 118/120, CUTEst 560/727 at v0.7.0 now 556/727 at v0.7.1 after adopting the Ipopt s_d/s_c scaling; see Section [CUTEst Benchmark Suite](#) and the electrolyte and grid domains) showed that once the barrier subproblem has been driven to a small μ and the post-restoration multipliers are reset cleanly (see Section [Two-Phase Restoration with Multi-Attempt Recovery](#)), the iterative z matches Ipopt’s `curr_dual_infeasibility` closely enough to decide convergence on its own. The auxiliary z_{opt} machinery was removed in v0.7.0. The current convergence test is the same two-gate check Ipopt 3.14.x uses.

4.2 Two-Gate Test on Iterative Multipliers

At each iteration, convergence is declared when **both** gates pass:

Scaled gate. Following Ipopt 3.14.x, ripopt computes *two* independent scale factors—one for the dual-stationarity term and one for the complementarity term:

$$s_d = \max\left(s_{\max}, \frac{\|y\|_1 + \|z_L\|_1 + \|z_U\|_1}{n + m}\right) / s_{\max}, \quad s_c = \max\left(s_{\max}, \frac{\|z_L\|_1 + \|z_U\|_1}{n}\right) / s_{\max}, \quad (8)$$

with $s_{\max} = 100$. The primal infeasibility is compared against the user tolerance ϵ directly; the dual infeasibility is compared against $\epsilon \cdot s_d$; the complementarity is compared against $\epsilon \cdot s_c$. Earlier ripopt releases used a single s_d for both terms and capped it at 10^4 to guard against runaway scaling. v0.7.1 separated s_d and s_c and dropped the cap, matching Ipopt’s `IpoptCalculatedQuantities::curr_nlp_error` exactly. The cap removal had a real effect on the CUTEst suite—some iterates that previously declared **Optimal** early on a loose scaled metric now run past their numerically-clean stopping point, and a small number of least-squares problems flipped from **Optimal** to **NumericalError** as a consequence (see Section [CUTEst Benchmark Suite](#)).

Unscaled gate. The raw residuals are compared against `constr_viol_tol`, `dual_inf_tol`, and `compl_inf_tol`. The unscaled gate is the safety net that catches cases where a large s_d or s_c would otherwise mask a non-converged iterate.

Both gates use the iterative z values directly. An acceptable-level state (mirroring Ipopt’s `acceptable_tol = 10^{-6}`, 15 consecutive iterations, relaxed unscaled tolerances of 10^{-2}) is tracked internally for best-duality bookkeeping, but ripopt returns **Optimal** only when the strict two-gate test passes at the user tolerance.

5 Two-Phase Restoration with Multi-Attempt Recovery

5.1 Background

When the filter line search^{5,4} fails to find an acceptable step at any step size $\alpha \geq \alpha_{\min}$, the solver must recover feasibility. In Ipopt, this triggers an NLP restoration phase that formulates and solves a full barrier subproblem minimizing constraint violation. If the restoration phase itself fails (e.g., the feasibility subproblem is locally infeasible), Ipopt terminates with `Restoration_Failed`. This is the dominant failure mode in practice—on the CUTEst suite, 32% of Ipopt’s failures are `Restoration_Failed`.

5.2 Soft Restoration

Before invoking either Phase 1 (Gauss-Newton) or Phase 2 (NLP) restoration, ripopt attempts a *soft restoration step*, a port of Ipopt’s `MinC_1NrmRestorationPhase::TrySoftRestoStep`.⁴ The idea is that when the line search fails on an iterate that is still close to feasibility, the cheapest viable recovery is often the same primal-dual Newton direction the IPM already computed, accepted on a relaxed E_μ criterion rather than the filter:

$$E_\mu(x, y, z) = \max\left(\frac{\|\nabla \mathcal{L}\|_\infty}{s_d}, \|c(x)\|_\infty, \frac{\|XZe - \mu e\|_\infty}{s_c}\right) \quad (9)$$

The trial point (x_+, y_+, z_+) is accepted as a soft-restoration step if $E_\mu(x_+, y_+, z_+) < (1 - \kappa_{\text{soft}}) E_\mu(x, y, z)$ with $\kappa_{\text{soft}} = 0.999$, i.e., any decrease in the barrier-KKT residual at the current μ is sufficient. Soft restoration is bypassed on iterates where $\theta(x) > \theta_{\max}$ or where the predicted decrease is dominated by complementarity (in which case μ should be reduced rather than restoration entered). Successful soft-restoration steps update the iterate in place and continue the main IPM loop without a filter reset; only when soft restoration fails does control fall through to Phase 1.

5.3 Phase 1: Gauss-Newton Feasibility Solver

Before invoking the expensive NLP restoration, we attempt a Gauss-Newton (GN) solve on the constraint violation (see¹¹):

$$\min_x \frac{1}{2} \|h(x)\|^2, \quad h_i(x) = \max(g_{L,i} - g_i(x), g_i(x) - g_{U,i}, 0) \quad (10)$$

The GN step solves the normal equations $(J_a J_a^T + \epsilon I)w = h_a$ where J_a is the Jacobian restricted to violated constraints, using Levenberg-Marquardt^{12,13} regularization with adaptive ϵ (starting at 10^{-8} , increasing by $10\times$ on factorization failure). The step is $\Delta x = -J_a^T w$, a minimum-norm solution in the full variable space.

The GN phase includes three fallback mechanisms:

1. **Gradient descent fallback:** When the GN line search fails to find sufficient decrease after 30 halvings, a steepest-descent step $\Delta x = -J^T h$ is computed with a separate

backtracking line search. This is critical for problems where the GN Hessian is nearly singular (e.g., when the Jacobian is rank-deficient).

2. **Proximity regularization:** When progress stalls (consecutive line search failures), a pull toward the starting point is added: $\Delta x_i \leftarrow \eta(x_{\text{rest},i} - x_i)$ with $\eta = 10^{-4}(1 + k_{\text{fail}})$ increasing with consecutive failures. This prevents the GN solver from wandering far when it cannot make progress.
3. **Penalty-regularized fallback:** If the main GN loop fails to achieve sufficient violation reduction (less than 50% or less than 1%), a penalty-regularized gradient descent minimizes $\frac{1}{2}\|h\|^2 + \rho\|x - x_{\text{ref}}\|^2$ with increasing $\rho \in \{10^{-6}, 10^{-4}, 10^{-2}\}$.

The GN phase provides quadratic convergence for nonlinear equalities (vs. linear for gradient descent) and handles 90%+ of restoration calls at a fraction of the cost of a full NLP subproblem.

5.4 Phase 2: NLP Restoration

If GN restoration fails on two consecutive occasions, we invoke the full NLP restoration subproblem, matching Ipopt’s formulation (see⁴):

$$\min_{x,p,n} \rho \sum_i (p_i + n_i) + \frac{\eta}{2} \|D_R(x - x_R)\|^2 \quad \text{s.t.} \quad g(x) - p + n = g_{\text{target}}, \quad p, n \geq 0 \quad (11)$$

where $\rho = 1000$, $\eta = \sqrt{\mu}$, D_R is a diagonal scaling matrix with $D_{R,i} = 1/\max(1, |x_{R,i}|)$, and x_R is the reference point (the iterate at restoration entry). This is solved by the same IPM engine with two safeguards against infinite recursion: (i) dynamic dispatch (`&dyn NlpProblem` in Rust) breaks the monomorphization chain, and (ii) `disable_nlp_restoration = true` in the inner solve prevents further nesting.

Critically, when the NLP restoration itself returns `LocalInfeasibility` or fails to converge, we do **not** immediately terminate. Instead, the result falls through to the multi-attempt recovery mechanism described below. This design reflects our observation that the stationarity-based infeasibility detection (computed at recovery attempt 7) is more reliable than the NLP restoration’s own convergence signal, which can declare local infeasibility prematurely.

5.5 Multi-Attempt Recovery

When both GN and NLP restoration fail, we cycle through up to 6 recovery attempts before declaring infeasibility:

Attempt	Strategy	Rationale
1	Mode switch (Free $\rightarrow$ Fixed, or force μ decrease)	Different barrier strategy may help
2	NLP restoration + $\mu \times 10$	Large μ smooths barrier landscape
3	$\mu \times 0.1 + x$ perturbation	Small μ may find narrow feasible path
4	$\mu \times 100 + x$ perturbation	Much larger μ for broader search
5	$\mu \times 0.01 + x$ perturbation	Much smaller μ for tight convergence
6	$\mu \times 1000 + x$ perturbation	Extreme μ increase as last resort

The μ factors $[10, 0.1, 100, 0.01, 1000, 0.001]$ alternate between increasing and decreasing by progressively larger magnitudes. This systematic exploration of different barrier landscapes is entirely novel compared to Ipopt, which terminates after a single restoration failure.

The x perturbation (attempts 3–6) uses a deterministic pattern $\Delta x_i = \text{sign}_i \cdot 10^{-4} \cdot \text{range}_i$ where $\text{sign}_i = (-1)^{[(7i+13k) \bmod 3=0]}$ depends on both the variable index i and the attempt number k , and $\text{range}_i = x_{U,i} - x_{L,i}$ for bounded variables or $\max(|x_i|, 1)$ for unbounded. This escapes the current basin while maintaining reproducibility.

After each *failed* recovery attempt, the filter is fully reset (all entries cleared, $\theta_{\max}$ recomputed from the current point) and the inertia-correction δ_w history is cleared, giving the algorithm a clean slate. (This is distinct from the post-*success* handoff in the next subsection, which deliberately retains the augmented filter entries.)

After all 6 attempts, infeasibility is checked via stationarity of the constraint violation function: if $\|J^T h(x)\|_\infty < 10^{-4} \cdot \max(\theta, 1)$, where $\theta = \|h(x)\|$ is the constraint violation, we report **LocalInfeasibility**—the iterate is at a stationary point of the violation function, indicating the problem is (locally) infeasible.

5.6 Restoration Convergence: Three-Gate Test

Inside the Phase 2 NLP restoration, the inner IPM uses a three-gate convergence test that ports Ipopt’s **RestoFilterConvCheck**.⁴ The plain barrier-KKT convergence used by the outer IPM is not the right termination criterion for restoration: restoration succeeds when (i) the original-problem filter is willing to accept the restored iterate, *or* (ii) the restored iterate satisfies the original problem’s tolerance, *or* (iii) the restoration NLP itself converges to a stationary point of $\|h\|^2$. The three gates are evaluated in this order and any one of them terminates restoration:

1. **Filter acceptance gate.** If the restored iterate is filter-acceptable for the *original* problem (i.e., it satisfies the same f/h -type acceptance conditions used by the outer line search), restoration returns success and the outer IPM resumes from this point with no further work.
2. **Original-problem KKT gate.** If the original-problem barrier-KKT error at the restored iterate—computed using the iterative bound multipliers reset to μ/s —is below the user tolerance, restoration returns **Optimal** directly.
3. **Restoration stationarity gate.** If the restoration NLP itself reaches its inner tolerance (i.e., the perturbed problem (11) is solved), restoration returns success and the

outer IPM resumes from the restored iterate after a state reset.

Earlier ripopt releases checked only gate (3) and missed many cases where the cheaper gates would have terminated restoration faster (and, more importantly, with a primal point that was directly usable by the outer IPM). The three-gate port closed several silent failures at the boundary between restoration and the main IPM, particularly on problems where the restoration NLP would converge to a slightly different basin than the original problem and the outer IPM would then fail to make progress from the restored iterate.

5.7 Post-Restoration State Handoff

When NLP restoration produces a recovered primal point x_+ , the parent IPM must reconstruct dual variables (y, z_L, z_U, v_L, v_U) that are usable by the next outer iteration. Earlier ripopt releases performed an aggressive heuristic reset (e.g., $z_i = \mu/s_i$ capped at 10^3 , $y = 0$, μ recomputed from average complementarity, filter cleared, mode forced to Free), but this reset interacted poorly with restoration cycles: on probe problems such as `arki0003` the heuristic inflated μ to $\sim 10^4$ and produced a near-unbarriered Newton system that immediately re-triggered restoration. The v0.8 handoff matches Ipopt's `MinC_1NrmRestorationPhase::PerformRestoration` line-for-line and is structured around four observations:

1. **Slacks first.** After x_+ replaces the parent's x , s is reinitialized so that $s_i \in [g_{L,i}, g_{U,i}]$ honors the new constraint values; without this push, $\theta(x_+, s)$ can still report the pre-restoration violation and the next iteration enters restoration again.
2. **Bound multipliers via a synthetic Newton step.** All four blocks (z_L, z_U, v_L, v_U) are updated by the closed-form step
3. **Equality multipliers y default to zero.** With Ipopt's default `=constr_mult_reset_threshold = 0.0` the LS estimator branch in `DefaultIterateInitializer::least_square_mult` is bypassed and $y_c = y_d = 0$ unconditionally. ripopt mirrors this default; with a positive threshold we keep the LS estimate when $\|y_{LS}\|_\infty \leq \text{threshold}$ and zero it otherwise. The LS estimate at a poorly-scaled restored iterate biases the parent's first Newton direction, which on `arki0003` produced a post-handoff dual-residual blow-up; zeroing y avoids this bias.
4. **No filter reset, no μ recompute.** The filter is *not* cleared after a successful restoration: the entries augmented at restoration entry are retained so that future iterations cannot revisit the basin restoration just escaped (only the `consecutive_acceptable` counter is zeroed). The barrier parameter μ is inherited from the pre-restoration value (clamped to $\mu_{\min}$), per `IpIpoptAlgorithm.cpp:842`; recomputing μ from the post-handoff average complementarity is exactly what inflated μ in the legacy reset because the nuclear-reset branch makes "fake" complementarity $1 \cdot s$ dominate the average. The mu-mode bookkeeping (`ref_vals`, `first_iter_in_mode`) is cleared so the standard adaptive/monotone update logic decides the next μ once the parent loop resumes.

The KKT factor cache and per-block atags are dropped at the boundary because x, y, z, s, v all change wholesale; this is the coarse "invalidate at boundary" path in lieu of fine-grained

per-write atag plumbing. The `resto_z` argument carried by older signatures is unused under this handoff and retained only for source compatibility during the v0.8 transition.

This handoff is the single largest correctness change in v0.8 relative to v0.7 restoration: every step matches a specific Ipopt reference site, and the post-restoration cycling that motivated the legacy heuristic resets is now absent on the alignment probe set.

6 Preprocessing

Before invoking the interior point solver, ripopt applies preprocessing to simplify the problem. v0.8 splits the preprocessing surface into two paths to match Ipopt 3.14’s `TNLAdapter` behavior more closely:

- `=fixed_variable_treatment = make_parameter=` (the default in v0.8, flipped from `relax_bounds` in earlier releases) eliminates fixed variables only and is the path Ipopt’s `TNLAdapter` takes.
- `=enable_preprocessing = true=` adds two ripopt-specific reductions on top of fixed-variable elimination—redundant constraint detection and bound tightening from linear constraints—which Ipopt does not do. These are queued for deletion in v0.9 ([Section Opportunities for Improvement](#)) but remain on by default in v0.8.

6.1 Fixed Variable Elimination

When $x_{L,i} = x_{U,i}$ (within tolerance 10^{-10}), variable x_i is fixed at $(x_{L,i} + x_{U,i})/2$ and eliminated from the optimization. The `PreprocessedProblem` wrapper presents a reduced problem with remapped variable indices, delegates all function evaluations to the original problem (inserting fixed values into the full-dimensional x), and remaps the sparsity patterns of the Jacobian and Hessian. After solving the reduced problem, the solution is unmapped back to the original dimension. Following Ipopt 3.14’s `TNLAdapter`, the wrapper commits to the reduced problem: there is no retry-on-failure with the unreduced problem if the IPM does not converge on the reduced one. Earlier ripopt releases (v0.7 and prior) did retry on failure; that fallback was removed in v0.8 once it was confirmed to suppress signal from genuine reduction-induced ill-conditioning rather than to recover useful solves.

6.2 Redundant Constraint Detection (legacy, default-on)

When `=enable_preprocessing = true=`, two constraints are considered redundant if they have identical bounds and identical Jacobian entries (same column structure and values) at two distinct evaluation points. Checking at two points prevents false matches from nonlinear constraints that happen to have the same Jacobian at a single point. Redundant constraints are removed from the reduced problem. This step has no Ipopt analogue; the audit list flags it for deletion in v0.9.

6.3 Bound Tightening from Linear Constraints (legacy, default-on)

When `=enable_preprocessing = true=`, for verified linear constraints (Jacobian unchanged between two evaluation points) with exactly one free-variable nonzero entry a_j , the constraint $g_L \leq a_j x_j + c \leq g_U$ yields tighter bounds on x_j :

$$\frac{g_L - c}{a_j} \leq x_j \leq \frac{g_U - c}{a_j} \quad (\text{if } a_j > 0) \quad (12)$$

where c is the fixed-variable contribution. This tightening is only applied after verifying constraint linearity, avoiding incorrect bounds from nonlinear constraints whose Jacobian happens to have a single nonzero at the initial point. Like the redundant-constraint pass, this step has no Ipopt analogue and is queued for deletion in v0.9.

7 NLP Scaling

riopt applies two layers of NLP scaling, both modeled on Ipopt 3.14’s `IpScaledNLP` / `IpStandardScalingBase` pipeline. The objective and each constraint row are scaled automatically by the gradient-based heuristic `=nlp_scaling_method = gradient-based=`: the factor for the objective is $\min(1, 100/\|\nabla f(x_0)\|_\infty)$, clamped from below at 10^{-2} , and the factor for constraint i is $\min(1, 100/\|J_i(x_0)\|_\infty)$. These scalings are computed once at the initial point and then folded into every objective, gradient, Jacobian, and Hessian evaluation through the same buffer-filling `NlpProblem` trait, so the IPM sees a problem in which no row of ∇f or J exceeds magnitude 100.

A separate *variable scaling* wrapper, `XScaledProblem`, was added in v0.7.1 to mirror the user-supplied side of Ipopt’s `IpScaledNLP`. When the caller passes `SolverOptions::user_x_scaling` a strictly-positive, finite vector d_x of length n , ripoft rewrites the IPM’s coordinate system as $x' = D_x x$ (with $D_x = \text{diag}(d_x)$) for the duration of the solve. Bounds, the gradient, the Jacobian, and the Hessian are transformed consistently, and the returned solution is unscaled before it leaves the public API: $x_{\text{user}} = x'/d_x$, $z_L^{\text{user}} = d_x \cdot z_L^{\text{internal}}$, $z_U^{\text{user}} = d_x \cdot z_U^{\text{internal}}$; constraint multipliers and constraint values are scale-invariant. Objective and constraint scaling are independent (`user_obj_scaling` and `user_g_scaling` remain available), so the three scalings can be combined or used in isolation. Invalid input—wrong length, non-positive, or non-finite entries—is rejected with `InternalError` rather than silently ignored, matching the strictness of Ipopt’s option validation. The `None` and `Some(vec![])` cases pass through to the automatic gradient-based scaling so that existing callers see no behavior change.

8 Near-Linear Constraint Detection

After scaling, ripoft evaluates the Jacobian at two distinct points (the initial point x_0 and a perturbed x_1) and compares entries per constraint row. If all entries for constraint i change by less than 10^{-12} (relative), the constraint is marked as linear. For linear constraints, $\nabla^2 g_i = 0$, so their $\lambda_i \nabla^2 g_i$ Hessian contribution is exactly zero. The solver zeros λ_i for detected linear constraints before calling the Hessian evaluation, avoiding unnecessary computation.

The perturbation $x_{1,j} = x_{0,j} + 1 + 0.1|x_{0,j}|$ is clamped to bounds, with fallback to the opposite direction if clamped back. Constraints involving variables that could not be perturbed by at least 10^{-4} are conservatively marked as nonlinear.

9 Multi-Solver Fallback Architecture

When the primary interior point method fails—returning `RestorationFailed`, `MaxIterations`, or `NumericalError`—riopt automatically attempts up to three alternative solvers before declaring failure. This multi-solver fallback architecture is a key contributor to ripoft’s robustness, recovering many problems that the primary IPM alone cannot solve.

9.1 L-BFGS for Unconstrained Problems

For unconstrained problems ($m = 0$, no finite bounds), a limited-memory BFGS (L-BFGS) solver¹¹ is tried **before** the IPM, since the barrier machinery is unnecessary when there are no constraints or bounds to enforce. L-BFGS maintains a compact representation of the inverse Hessian approximation using the most recent $k = 10$ curvature pairs, requiring only $O(nk)$ storage and $O(nk)$ per-iteration cost. The line search uses Wolfe conditions with backtracking.

For bound-constrained problems ($m = 0$, finite bounds), L-BFGS is used as a **fallback** after IPM failure, with a projected gradient approach that clamps iterates to the feasible box at each step.

The L-BFGS solver is particularly effective for problems where the exact Hessian is pathological far from the solution (e.g., strongly indefinite), as the quasi-Newton approximation builds up curvature information incrementally. On the CUTEst suite, L-BFGS solves several unconstrained problems (PALMER5A, PALMER7E) where the IPM’s inertia correction struggles.

9.2 Augmented Lagrangian for Constrained Problems

For constrained problems that fail the primary IPM, ripoft applies a penalty-based augmented Lagrangian (AL) method using the Powell-Hestenes-Rockafellar (PHR) formulation.¹¹ The AL subproblem at outer iteration k is:

$$\min_x f(x) + \sum_i \left[y_i^{(k)} h_i(x) + \frac{\rho^{(k)}}{2} h_i(x)^2 \right] \quad (13)$$

where $h_i(x)$ is the constraint violation (accounting for inequality slacks), $y_i^{(k)}$ are the Lagrange multiplier estimates, and $\rho^{(k)}$ is the penalty parameter. Each AL subproblem is an unconstrained (or bound-constrained) NLP solved by the same IPM engine.

The multiplier update follows the standard rule $y_i^{(k+1)} = y_i^{(k)} + \rho^{(k)} h_i(x^*)$, and the penalty parameter increases by a factor of 10 when constraint violation fails to decrease sufficiently. The AL method converges when the constraint violation and KKT error both fall below tolerance.

The AL fallback is effective for problems where the IPM’s filter line search or restoration phase fails but the feasible region is reachable via penalty methods. On CUTEst, this recovers several problems (CRESC50, TRO6X2) that resist the primary IPM.

9.3 SQP Fallback for Constrained Problems

For constrained problems where both the IPM and AL fail, ripopt tries a Sequential Quadratic Programming (SQP) solver. At each iteration, the SQP solver assembles and solves the $(n + m) \times (n + m)$ KKT system of the QP subproblem:

$$\begin{bmatrix} H & J^T \\ J & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \lambda_{\text{new}} \end{bmatrix} = \begin{bmatrix} -\nabla_x \mathcal{L} \\ -c(x) \end{bmatrix} \quad (14)$$

where H is the Hessian of the Lagrangian, J is the constraint Jacobian, and $c(x)$ is the constraint residual. Inertia correction is applied when the KKT factorization has incorrect inertia (n positive, m negative eigenvalues required). A backtracking line search with an ℓ_1 merit function $\phi(x) = f(x) + \rho \|c(x)\|_1$ determines the step size, with $\rho = \max(|\lambda_{\text{new}}|) + 1$. The SQP result replaces the current best only if it is strictly better (higher solve status or lower objective at the same status).

9.4 Explicit Slack Reformulation

For constrained problems with inequality constraints that fail both the primary IPM and the AL fallback, ripopt reformulates the problem with explicit slack variables:

$$\min_{x,s} f(x) \quad \text{s.t.} \quad g(x) - s = 0, \quad g_L \leq s \leq g_U \quad (15)$$

This converts inequality constraints into equalities with bounded slacks, producing a formulation closer to Ipopt’s native representation. The reformulated problem is solved by the same IPM engine with `enable_slack_fallback = false` to prevent recursive fallback.

The explicit slack formulation addresses a fundamental limitation of the implicit slack approach: at degenerate points where constraint multipliers oscillate (Section [Handling Infeasible Iterates](#)), explicit slacks provide well-behaved iterative multiplier updates that stabilize convergence. This fallback solves problems like HATFLDH and HS83 where restoration cycling in the implicit-slack formulation prevents convergence.

9.5 Best-Point Tracking for Cycling Detection

Throughout the main IPM solve, ripopt tracks the iterate with the lowest dual infeasibility encountered during the solve. When the solver reaches the maximum iteration limit, it restores this best-dual point and checks whether it satisfies the acceptable convergence criteria. This mechanism recovers problems where the solver cycles between near-optimal points without satisfying the strict convergence test—for example, HAHN1LS, where the dual infeasibility cycles between 1.46 and 4.25×10^{-3} but the best point achieves acceptable convergence.

The primal tolerance at the max-iteration exit is relaxed to $\max(\epsilon_{\text{acc}}, \epsilon_{\text{cv,acc}})$, where $\epsilon_{\text{cv,acc}}$ is the acceptable constraint violation tolerance. This allows problems like SSINE, where constraint violation stalls at 1.4×10^{-3} after 3000 iterations, to be accepted when all other convergence criteria are met.

10 Adaptive Barrier Parameter

10.1 Oracles

riopt provides two adaptive- μ oracles, both ported faithfully from Ipopt (see^{6,4}):

LOQO oracle (default, `=mu_oracle = loqo=`). Mirrors Ipopt's `LoqoMuOracle::CalculateMu`.¹⁴ The centering parameter is computed from the centrality measure $\xi = \min_i(s_i z_i)/\bar{c}$:

$$\sigma = 0.1 \cdot \min\left(\frac{0.05(1 - \xi)}{\xi}, 2\right)^3, \quad \mu = \text{clamp}(\sigma \bar{c}, \mu_{\min}, \mu_{\max}) \quad (16)$$

where $\bar{c}$ is the average complementarity over all variable-bound and constraint-slack pairs and $\mu_{\max} = \kappa_{\mu_{\max}} \cdot \bar{c}_0$ is captured lazily on the first oracle call (Ipopt's `mu_max_factor` in `initial_avg_compl`, `IpAdaptiveMuUpdate.cpp:267-273`). Earlier rlopt releases used a simpler $\mu = \bar{c}/\kappa$ with $\kappa = 10$; we replaced it with this exact LOQO formula in v0.8 because the simpler version monotonically picks $\sigma = 0.1$ regardless of centrality and tends to over-center on highly off-central iterates produced by restoration handoffs.

Quality-Function oracle (optional, `=mu_oracle = quality_function=`). Mirrors `IpQualityFunctionMuOracle.cpp:154-485`. At each iteration we factor the augmented KKT, solve the affine-predictor RHS to obtain d_{aff} (the $\mu = 0$ direction) and the full RHS at μ_{cur} to obtain d_{full} , then evaluate

$$Q(\sigma) = \frac{(1 - \alpha_d) \text{dual}_0}{s_d} + (1 - \alpha_p) \text{primal}_0 + \frac{\max_i(s_i^+ z_i^+)}{s_c} + (1 - \min(\alpha_p, \alpha_d)) \cdot \frac{\max(\bar{c}, \mu)}{s_c} \quad (17)$$

where the trial step at candidate σ is $d(\sigma) = (1 - \sigma) d_{\text{aff}} + \sigma d_{\text{full}}$ (which by linearity solves the KKT system at $\mu_{\text{target}} = \sigma \mu_{\text{cur}}$) and (α_p, α_d) are taken with $\tau = 1$. An optional centrality penalty ν/ξ^+ is added when `quality_function_centrality` is set. The minimisation is a golden-section search over $\sigma \in [10^{-6}, 1]$ with at most 8 steps and relative bracket tolerance 10^{-2} . On factor or solve failure, the call returns `None` and the caller falls back to the LOQO oracle, exactly as Ipopt skips the candidate.

10.2 Free and Fixed Modes

Following Ipopt's `IpAdaptiveMuUpdate::DoUpdate`, the barrier parameter alternates between two modes:

- **Free mode** runs the configured μ -oracle whenever sufficient progress holds, then resets the filter at the new μ (per `IpFilterLSAcceptor.cpp:524-532`). "Sufficient progress" is the standard test on a sliding window of ≤ 4 KKT-error reference values; with fewer

than 4 entries the test returns true unconditionally, so the earliest possible Free→Fixed switch is iteration 4 (matching Ipopt). When the test fails or a tiny step is detected, Free mode switches to Fixed and seeds $\mu_{\text{new}} = \kappa_{\text{init}} \cdot \bar{c}$, clamped to $[\mu_{\text{min}}, \mu_{\text{max}}]$.

- **Fixed mode** decreases μ monotonically: when the barrier-KKT error drops below $\kappa_\epsilon \cdot \mu$, we apply $\mu_{\text{new}} = \min(\kappa_\mu \mu, \mu^{\theta_\mu})$ with $\kappa_\mu = 0.2$ and $\theta_\mu = 1.5$. The mode switches back to Free when an adaptive strategy is configured and the Fixed-mode subproblem is solved on a non-first iteration.

riopt previously had a third "conservative decrease" branch and a `barrier_subproblem_solved` gate that survived in Free mode; both were deleted in v0.8 because Ipopt 3.14 has no analogue (Free mode either runs the oracle or switches to Fixed; the κ_ϵ gate is Fixed-mode-only).

10.3 Additional Design Choices

Fraction-to-boundary rule. The step-length bound τ uses $\tau = \max(\tau_{\text{min}}, 1 - e_\mu)$ in Free mode, where e_μ is the barrier-KKT error at the current μ , and Ipopt's standard $\tau = \max(\tau_{\text{min}}, 1 - \mu)$ in Fixed mode. The Free-mode form is more aggressive near convergence ($e_\mu \rightarrow 0 \Rightarrow \tau \rightarrow 1$) and matches the rule used by Ipopt's adaptive strategy.

No-bounds special case. When no variable has finite bounds, μ is set to $\mu_{\text{min}} = 10^{-11}$ immediately. With no bound constraints the average complementarity is zero and the barrier parameter serves no purpose; setting it to μ_{min} avoids stalling on iterates where $\bar{c} = 0$ would otherwise short-circuit the oracle.

Warm-start μ override. When rlopt is invoked with primal/dual warm-start data from a previous solve, the cold-start ramp on μ (initial $\mu_0 = 0.1$ followed by several iterations of decrease) is wasted work—the supplied iterate is, by assumption, already close to the central path at some smaller μ . The `warm_start_target_mu` option directly initializes μ_0 to the value the prior run finished at (or any user-supplied value), bypassing the cold-start schedule. This is the same mechanism Ipopt's `warm_start_mult_init_max` option provides, and is essential for parametric studies and SIPOPT-style sensitivity sweeps where successive solves share all but a small parameter perturbation.

11 Filter Line Search

We use the filter line search of⁵ as adapted by,⁴ with the following implementation details.

11.1 Switching Condition

The switching condition determines whether the current iteration uses f -type (objective decrease) or h -type (constraint violation decrease) acceptance:

$$\alpha(-\nabla\phi^T d)^{s_\phi} > \delta \theta^{s_\theta} \quad \text{and} \quad \theta < \theta_{\text{min}} \quad (18)$$

The inclusion of α as a multiplier on the left-hand side is critical: as α decreases during backtracking, the switching condition eventually fails, allowing the step to be accepted by

h -type criteria (constraint reduction). An early version of ripopt omitted the α factor, causing the switching condition to remain active even at tiny step sizes and blocking h -type acceptance.

11.2 Second-Order Correction

We implement iterative second-order correction (SOC)¹¹ to address the Maratos effect, where the constraint linearization error causes rejection of good Newton steps. Up to $k_{\text{soc}} = 4$ SOC iterations are performed, each solving the same factored KKT matrix with a modified right-hand side that accounts for the nonlinear constraint residual at the trial point. A convergence check $\theta_{\text{soc}} \geq \kappa_{\text{soc}} \theta_{\text{prev}}$ with $\kappa_{\text{soc}} = 0.99$ terminates SOC early when constraint violation is not improving.

11.3 Objective Safeguard

Before checking filter acceptability, we reject any trial point where the barrier objective increases catastrophically:

$$\phi_{\text{trial}} > \phi_{\text{current}} + c_{\text{obj}}(1 + |\phi_{\text{current}}|) \quad (19)$$

with $c_{\text{obj}} = 5$ (Ipopt's `obj_max_inc`). Per the v0.8 alignment with `IpFilterLSAcceptor.cpp:280-371`, this divergence guard is *h-type only*: Ipopt's f -type Armijo branch never consults `obj_max_inc`, and the previous ripopt convention that applied the guard on every trial would silently reject valid Armijo steps near the central path. The h -type branch evaluates the guard before any filter-pair test.

11.4 Filter Augmentation and Acceptance Convention

The filter is augmented with the pair (θ_F, ϕ_F) defined by Ipopt's `AugmentFilter` rule

$$\theta_F = (1 - \gamma_\theta) \theta(x), \quad \phi_F = \phi(x) - \gamma_\phi \theta(x), \quad (20)$$

i.e., the existing point's pair shrunk by the same envelope used by acceptance. The $\leq$ comparisons in `Compare_le` use $|\theta_F| \cdot \text{eps}_{\text{mach}}$ as a tolerance; ripopt previously used a stricter exact comparison that double-counted the envelope and rejected genuinely acceptable trials when the post-restoration handoff produced a θ near the augmented entry. Both fixes are recorded in commits `acf87f5` and `c72802b`. Restoration entries are bypassed during augmentation (Ipopt's `RestoFilterConvCheck` does not augment) so that successful inner restoration does not poison the parent filter.

12 Linear Algebra

12.1 Bunch-Kaufman Factorization

For small-to-medium problems ($n + m < 100$), we use a custom dense LDL^T factorization with Bunch-Kaufman pivoting.¹⁵ This produces the factorization $\text{PLDL}^T P^T = A$ where P

is a permutation, L is unit lower triangular, and D is block diagonal with 1×1 and 2×2 blocks. The 2×2 blocks handle the indefinite structure of the KKT matrix without requiring positive definiteness.

A correctness issue in many textbook implementations of Bunch-Kaufman is that when rows and columns are swapped during pivoting, the L entries from previously computed columns must also be swapped. Without this, $PLDL^T P^T \neq A$, producing incorrect solutions that can be difficult to diagnose (the factorization appears to succeed but the solve returns wrong search directions).

12.2 Sparse LDL^T Factorization

For larger problems ($n+m \geq 100$), ripopt uses **feral**,¹ a pure Rust multifrontal LDL^T solver with Bunch-Kaufman $1 \times 1/2 \times 2$ pivoting, certified inertia counts, MC64 matching-based scaling, AMD and METIS nested-dissection orderings, and built-in iterative refinement. The multifrontal method organizes the factorization into dense operations on frontal matrices defined by a supernodal elimination tree, providing better cache utilization than simplicial methods. Ordering and symbolic analysis are computed once and cached across IPM iterations; only the numeric factorization is repeated each iteration. The upper-triangle COO triplet store used by ripopt’s KKT assembly is converted to feral’s lower-triangle CSC by swapping $(i, j) \rightarrow (j, i)$, and a COO→CSC index mapping is cached so that inertia-correction re-factorizations scatter values in place and call the numeric phase only, avoiding repeated symbolic work and memory allocation.

A pure-Rust multifrontal solver in this position has two practical implications. First, ripopt builds with a single **cargo build** command on any platform with a Rust toolchain; there is no Fortran dependency, no separate BLAS install, no **cmake**, and no system package manager involvement. Second, feral exposes its internal scaling, pivot threshold, and zero-pivot policy through a typed API rather than a single integer-flag interface, which lets the wrapper drive a two-stage **increase_quality** cascade that mirrors Ipopt’s `IpTSymLinearSolver::IncreaseQuality`. Stage one flips the in-solver scaling from Identity to InfNorm; stage two raises the Bunch-Kaufman column-relative pivot threshold using an MA27-style schedule. Scaling is left at Identity by default because ripopt owns the KKT-level scaling decision through its own Ruiz equilibration path (Section [KKT-Aware Factorization for Indefinite Systems](#)); double-scaling at the linear-solver layer changes the factored matrix shape and corrupts the inertia signal that the IPM uses to drive its perturbation handler.

The legacy **rmumps** backend—an earlier pure-Rust multifrontal solver developed alongside ripopt and used as the default through v0.7—remains available behind the opt-in **rmumps** feature for A/B comparison and regression hunting. **rmumps** and **feral** present the same **LinearSolver** trait surface, so the IPM, KKT assembly, and restoration paths are oblivious to which backend is active. Backend-specific kernel details that affect numerical behavior are noted explicitly in the next subsection.

12.3 Inertia Correction

The KKT matrix (6) must have inertia $(n, m, 0)$ — n positive and m negative eigenvalues—for the Newton step to be a descent direction for the barrier problem.¹¹ After factorization, we

extract the inertia from the D factor (using eigenvalues of 2×2 blocks where present) and add diagonal regularization δ_w to the $(1, 1)$ block and $-\delta_c$ to the $(2, 2)$ block if the inertia is incorrect.

The regularization follows Ipopt’s `PDPerturbationHandler` escalation schedule.⁴ The first δ_w value attempted at each iteration is the smaller of $\bar{\delta}_w$ (the value that succeeded last time, decayed by a factor of $1/3$) and $\delta_{w,\min} = 10^{-20}$. On failure, δ_w is multiplied by a growth factor of 8 on the first escalation (matching Ipopt’s `delta_w_first_inc`) and by 4 on subsequent escalations, up to 15 attempts total. The $(2, 2)$ -block regularization is *scaled by the barrier parameter*, $\delta_c = \kappa_c \mu^{\alpha_c}$ with $\kappa_c = 10^{-8}$ and $\alpha_c = 0.25$ (Ipopt defaults). This -dependent schedule lets δ_c shrink as the barrier subproblem tightens, so the late-iteration KKT system is a faithful Newton system for the unperturbed problem rather than one biased by a fixed 10^{-4} floor; earlier ripopt releases used $\delta_c = 10^{-4}$ throughout.

For unconstrained problems ($m = 0$), we first try a direct computation $\delta_w = -d_{\min} + 10^{-8}$ where $d_{\min}$ is the minimum diagonal of D , avoiding the exponential search entirely. The inertia check itself is exact: the previous heuristic of accepting inertia within ± 1 of $(n, m, 0)$ on large systems was dropped in v0.7.1, once the rmumps contribution-block 2×2 pivot search and the feral force-accept-with-reinterpret pipeline (both described in Section [KKT-Aware Factorization for Indefinite Systems](#)) made the inertia signal trustworthy enough that the slack was hiding genuine inertia failures rather than absorbing classification noise.

A key design decision: when inertia correction fails after all 15 attempts, ripopt proceeds with the approximate factorization rather than returning an error. The filter line search will reject bad steps (producing small α), and restoration can recover. This is more pragmatic than Ipopt’s approach of immediately reporting `ErrorInStepComputation`, and allows problems like EQC and HIMMELBJ to converge where Ipopt fails.

12.4 Iterative Refinement

After each KKT solve, up to 3 rounds of iterative refinement are applied using the already-factored matrix (5 rounds for large systems or when inertia correction was applied). Convergence is checked in the ℓ_∞ norm with tolerance 10^{-12} , with stagnation detection for the inertia-correction path. When inertia correction was applied, refinement targets the *original* (unperturbed) system via an analytically recovered matrix-vector product, so that the Newton direction converges to the true solution despite the regularized factorization. The backward error threshold is 10^{-6} for small systems and 10^{-4} for large systems ($n + m \geq 100$) where sparse factorization accuracy is more limited.

12.5 KKT-Aware Factorization for Indefinite Systems

Symmetric indefinite KKT systems from constrained optimization pose specific numerical challenges for sparse LDL^T factorization. Equality constraints produce zero entries on the $(2, 2)$ block diagonal, creating near-zero pivots that cause catastrophic cancellation during elimination. Ipopt avoids this by relying on HSL MA57 or MUMPS—Fortran solvers with decades of refinement for indefinite systems. ripopt’s two pure-Rust backends each handle the indefinite case explicitly; this subsection describes the mechanisms relevant to KKT systems in each.

12.5.1 feral (default v0.8 backend)

The default backend is `feral`,¹ a pure-Rust multifrontal LDL^T solver designed from the start for symmetric indefinite systems. Three mechanisms are particularly relevant for KKT systems:

Bunch-Kaufman $1 \times 1/2 \times 2$ pivoting. Within each supernode, `feral` performs Bunch-Kaufman pivoting¹⁵ with column-relative pivot threshold u (MA27-style; default $u = 0$ for fastest factorization, escalated up to $u = 0.5$ via the `increase_quality` cascade described below). The 2×2 blocks naturally absorb the indefinite structure created by equality constraints, eliminating zero-diagonal rows in pairs with the primal variables they couple to.

Force-accepted near-zero pivots with IPM-driven inertia correction. The wrapper sets `ZeroPivotAction::ForceAccept` so `feral` keeps the factor available even when a near-zero pivot is hit; `riopt`'s `factor_with_inertia_correction` loop (Section [Inertia Correction](#)) then perturbs the diagonal and retries. This separation—`feral` reports an inertia signal, `riopt` drives the perturbation—matches `Ipopt`'s division of labor between `IpMa57TSolverInterface` and `PDPerturbationHandler`, and is more robust than embedding perturbation logic in the linear solver itself. To match the inertia semantics of MUMPS/MA57 (which never report a "zero" bucket because they delay or perturb tiny pivots), the wrapper reinterprets `feral`'s `zero` count as additional `negative` pivots for the regularized augmented system, where near-zero pivots in the Schur block carry an effective sign of $-\delta_c$.

Two-stage `increase_quality` escalation. When the inertia-correction ladder still cannot place the iterate within the trusted inertia $(n, m, 0)$, the wrapper escalates linear-solver quality in two stages, mirroring `IpTSymLinearSolver::IncreaseQuality`. Stage one flips the in-solver scaling from `Identity` to MC64-derived `InfNorm` scaling; stage two raises the Bunch-Kaufman column-relative pivot threshold using $u_{new} = u^{0.75}$ capped at $u_{max} = 0.5$. Scaling is left at `Identity` by default because `riopt` owns the KKT-level scaling decision (Ruiz equilibration is applied selectively to the assembled augmented system, not the raw Hessian-Jacobian blocks); double-scaling at the linear-solver layer changes the factored matrix shape and corrupts the inertia signal.

12.5.2 rmumps (legacy v0.7 backend)

The earlier `rmumps` backend—a separate pure-Rust multifrontal solver developed alongside `riopt` and used as the default through v0.7—remains available behind the opt-in `rmumps` feature. Several KKT-aware features were added to `rmumps` over the v0.7 series; they are documented here for completeness and because they remain in active use whenever the `rmumps` feature is enabled.

Threshold pivoting. Following MUMPS's configuration in `Ipopt` (`=CNTL(1) = 1e-6`), `rmumps` uses a very permissive pivot threshold $u = 10^{-6}$. A pivot a_{kk} is accepted as a 1×1 pivot if $|a_{kk}| \geq u \cdot \max_{j \neq k} |a_{jk}|$. This rarely triggers delayed pivoting, preventing the cascade of delayed pivots that previously caused factorization failures on medium-scale problems.

Genuine delayed pivoting. When a column fails the threshold test, it is genuinely delayed to the parent supernode (matching MA57/MUMPS behavior) rather than force-accepted with the largest available diagonal. The parent's frontal matrix is dynamically

expanded to accommodate delayed columns from children. This is strictly better than the force-acceptance fallback and can only improve factorization quality.

Contribution block pivot search. When all fully-summed columns in a front fail the threshold test, `rmumps` searches the contribution block for a 2×2 pivot partner. Columns are classified by diagonal magnitude: a zero-diagonal column (equality constraint) is paired with a non-zero-diagonal column (primal variable), forming a 2×2 block $\begin{pmatrix} H_{jj} + \Sigma_j & J_{ij} \\ J_{ij} & 0 \end{pmatrix}$ with determinant $-J_{ij}^2$ —always well-conditioned regardless of the zero diagonal. This is the key mechanism for KKT systems in this backend: it enables stable elimination of equality constraint variables that would otherwise produce catastrophic element growth. On the `camshape800` benchmark (1599 variables, 1600 constraints, 800 equalities), this increases the initial step length from $\alpha = 0.027$ to $\alpha = 0.99$.

Matching-based scaling. For KKT systems (identified by the `n_primal` parameter), `rmumps` applies a two-phase scaling before factorization: (1) normalize each row/column by $1/\sqrt{\max_j |a_{ij}|}$ so the largest entry per row is $O(1)$, then (2) Ruiz iterative equilibration for fine-tuning. This approximates MUMPS’s `=ICNTL(8) = 77=` automatic scaling and reduces the dynamic range across rows, improving pivot quality. (`feral`’s MC64 scaling plays the analogous role for the default backend.)

NEMIN supernode amalgamation. After fundamental supernode detection, consecutive small supernodes (fewer than `NEMIN = 4` columns) are merged into larger blocks. This creates larger frontal matrices with more columns available for pivot selection, matching the MA57/MUMPS approach of aggressive amalgamation for better BLAS-3 utilization and pivot quality.

KKT matching ordering. An optional ordering (`KktMatchingAmd`) computes a greedy maximum-weight matching on the Jacobian, pairing each constraint with its most-coupled variable. Matched pairs are merged into super-nodes in a compressed graph, AMD is applied to the compressed graph, and super-nodes are expanded back so paired columns are adjacent in the elimination order. This enables Bunch-Kaufman to naturally form 2×2 pivots at paired locations.

12.6 Nested Dissection Ordering

For large sparse systems, fill-reducing orderings are critical for factorization efficiency. `feral` ships METIS-style multilevel nested dissection¹⁶ alongside AMD as a default-available ordering, automatically selecting between AMD and ND based on graph size and density. `rmumps` implements its own multilevel nested dissection algorithm,¹⁷ available in addition to AMD: the algorithm recursively bisects the adjacency graph of the matrix into two subgraphs connected by a small separator, then orders the separator last (so it appears at the bottom of the elimination tree). Each bisection uses a multilevel Fiduccia–Mattheyses refinement scheme: the graph is coarsened via heavy-edge matching, bisected at the coarsest level using breadth-first search from a pseudo-peripheral node, and then uncoarsened with boundary refinement passes that optimize cut size subject to a balance constraint ($\leq 40\%$ imbalance). Recursion terminates at subgraphs below 200 nodes, where AMD ordering is applied. For problems with regular sparsity structure (e.g., finite-element meshes), nested

dissection produces $O(n \log n)$ fill versus $O(n^{3/2})$ for minimum degree.¹⁷

12.7 Iterative Solver: Preconditioned MINRES

For very large problems where even sparse direct factorization is too expensive, ripopt provides an iterative linear-solver path based on the MINRES algorithm of.⁹ MINRES is a Krylov subspace method that minimizes the residual norm $\|b - Ax\|_2$ for symmetric (possibly indefinite) systems—exactly the structure of the KKT system (6). Unlike CG (which requires positive definiteness) or GMRES (which requires nonsymmetric storage), MINRES exploits the symmetric indefinite structure directly with short recurrences, requiring only $O(n)$ storage beyond the matrix.

The iterative path is selected via the `=linear_solver = Iterative=` option and activates when $n + m \geq \text{sparse_threshold}$ (default 110). Below this threshold, the dense Bunch-Kaufman solver is used regardless of the `linear_solver` setting. Backend coverage of the iterative path differs by which sparse backend is enabled:

***rmumps backend (legacy).** The iterative path uses an incomplete LDL^T factorization^{10,18} as a preconditioner. The incomplete factorization reuses the same multifrontal elimination tree as the direct solver but applies dropping rules during each supernode’s partial factorization: entries in L_{21} and the Schur complement (contribution block) smaller than $\tau \cdot \|L_{:,j}\|$ are zeroed, where $\tau = 0.01$ is the drop tolerance. This produces a sparser approximate factorization $\tilde{L}\tilde{D}\tilde{L}^T \approx A$ that can be applied as a preconditioner in $O(\text{nnz}(\tilde{L}))$ time per MINRES iteration. The pivot threshold for Bunch-Kaufman pivoting is relaxed to 0.01 (vs. 0.5 for the direct solver) to allow more aggressive dropping. The inertia from the incomplete factorization is returned as an approximation of the true inertia; while not exact, it is sufficient for the IPM’s inertia correction to detect gross inertia errors (e.g., zero negative eigenvalues when $m > 0$) and add appropriate regularization.

***feral backend (default).** feral 0.1.0 does not yet expose an `IncompleteLdlT`-style preconditioner or a public MINRES driver. The intended design is to use the full feral factorization as the MINRES preconditioner, which makes the Krylov outer loop converge in one step and reduces algorithmically to plain iterative refinement—which `feral::solve_sparse_refined` already performs internally. The v0.8 wrapper therefore delegates `=linear_solver = Iterative=` to a thin wrapper around the direct path that tightens convergence expectations on the refinement loop. Under the feral backend, `Iterative` and `Direct` are algorithmically equivalent. A true incomplete- LDL^T + MINRES path under feral is deferred to v0.9.

12.8 Hybrid Direct-Iterative Solver

The hybrid solver combines the reliability of direct factorization with the scalability of iterative methods. It starts in direct mode (multifrontal LDL^T) and automatically switches to iterative mode under two conditions:

1. **Factorization failure:** If the direct factorization fails (e.g., numerical breakdown), the hybrid solver switches to iterative mode for subsequent iterations.
2. **Excessive factorization time:** If the direct factorization takes longer than a threshold (default 1.0 seconds wall-clock), the hybrid solver switches to iterative mode.

The hybrid solver also switches *back* to direct mode when the iterative solver fails: if MINRES does not converge within its iteration limit on 3 consecutive KKT solves, the solver reverts to direct factorization. This bidirectional switching prevents the solver from getting stuck in an ineffective mode.

The hybrid solver is selected via the `=linear_solver = Hybrid=` option. As with the iterative path, the practical behavior under the v0.8 feral backend reduces to "direct factorization with extra iterative refinement on retry" because feral does not yet expose a separate incomplete preconditioner; the crossover/timeout logic is preserved unchanged so the interface can be promoted to a true hybrid in a future release without changing call sites. Under the legacy `rmumps` backend, `Hybrid` is a fully bidirectional direct/MINRES switcher and is particularly effective for problems where the KKT system starts well-conditioned (direct factorization is fast) but becomes increasingly ill-conditioned as the barrier parameter decreases (making iterative methods more appropriate for the later, larger-regularization iterations).

12.9 Configurable Stall Detection

The IPM includes a stall detector that terminates the solve when insufficient progress is detected over a configurable window. The `stall_iter_limit` parameter (default 30) specifies the maximum number of consecutive iterations without at least 1% improvement in either the primal infeasibility or the dual infeasibility. When the stall limit is reached, the solver checks whether the current iterate satisfies the acceptable convergence criteria before declaring `MaxIterations`. This prevents the solver from spending thousands of iterations making negligible progress on problems with slow tails, while the configurable threshold allows users to trade patience for robustness on difficult problems.

13 Implementation

13.1 Language and Dependencies

riopt is implemented in Rust (approximately 21,700 lines of source code) with minimal external dependencies. The default v0.8.0 build pulls only the `feral` crate¹ for sparse multifrontal LDL^T factorization, the `faer` crate for dense linear algebra primitives, and the `log` / `serde` utility crates; the legacy `rmumps` multifrontal solver—a pure-Rust workspace-member crate that was the default through v0.7 and remains licensed separately under CeCILL-C—is preserved behind the opt-in `rmumps` Cargo feature for A/B comparison. `feral` ships AMD and METIS-style nested-dissection orderings,^{17,16} Bunch-Kaufman pivoting,¹⁵ MC64 matching-based scaling, and supernodal amalgamation; `rmumps` ships those plus a contribution-block 2×2 pivot search for KKT systems and an incomplete LDL^T preconditioner.¹⁰ The dense Bunch-Kaufman factorization, KKT assembly, filter, restoration, and all algorithmic logic are implemented from scratch. The codebase includes dedicated modules for the L-BFGS solver (`lbfgs.rs`), augmented Lagrangian method (`augmented_lagrangian.rs`), explicit slack reformulation (`slack_formulation.rs`), and direct/iterative/hybrid linear-solver wrappers (`linear_solver/feral_direct.rs`, `linear_solver/feral_iterative.rs`, `linear_solver/feral_hybrid.rs`, with parallel modules for the legacy `rmumps` path). The

library compiles to a single Cargo workspace with no C, C++, or Fortran dependencies in the default configuration, simplifying installation, cross-compilation, and embedding.

13.2 Problem Interface

The `NlpProblem` trait defines the interface for specifying optimization problems:

```

1 pub trait NlpProblem {
2     fn num_variables(&self) -> usize;
3     fn num_constraints(&self) -> usize;
4     fn bounds(&self, x_l: &mut [f64], x_u: &mut [f64]);
5     fn constraint_bounds(&self, g_l: &mut [f64], g_u: &mut [f64]);
6     fn initial_point(&self, x0: &mut [f64]);
7     fn objective(&self, x: &[f64]) -> f64;
8     fn gradient(&self, x: &[f64], grad: &mut [f64]);
9     fn constraints(&self, x: &[f64], g: &mut [f64]);
10    fn jacobian_structure(&self) -> (Vec<usize>, Vec<usize>);
11    fn jacobian_values(&self, x: &[f64], vals: &mut [f64]);
12    fn hessian_structure(&self) -> (Vec<usize>, Vec<usize>);
13    fn hessian_values(&self, x: &[f64], obj_factor: f64,
14                      lambda: &[f64], vals: &mut [f64]);
15 }
```

All evaluation methods use a buffer-filling pattern (the caller provides pre-allocated slices) to avoid heap allocation in the hot loop. The Jacobian and Hessian use sparse triplet format. The public API consists of a single function:

```

1 pub fn solve<P: NlpProblem>(problem: &P, options: &SolverOptions)
2     -> SolveResult;
```

This interface is intentionally similar to Ipopt’s TNLP interface, making it straightforward to port existing Ipopt problem implementations to ripopt.

13.3 Performance Characteristics

Rust’s ownership model and zero-cost abstractions provide several performance advantages:

- **Stack allocation:** All per-iteration working vectors are pre-allocated before the main loop. No heap allocation occurs during iteration.
- **No interop overhead:** Unlike Ipopt, which calls into Fortran (MUMPS) or C (MA27) for each factorization, ripopt’s linear algebra is in-process Rust code.
- **Cache efficiency:** The dense Bunch-Kaufman operates on contiguous memory with predictable access patterns.
- **No runtime overhead:** Rust has no garbage collector, no virtual machine, and no runtime type checking.

These advantages are most pronounced for small problems ($n + m < 50$) where per-iteration time is dominated by overhead rather than $O(n^3)$ factorization.

13.4 AMPL NL Interface and Pyomo Integration

To support integration with algebraic modeling languages, `riopt` includes a built-in AMPL NL file parser and SOL file writer. The `riopt` binary reads a problem in AMPL’s NL format,¹⁹ solves it, and writes the solution in SOL format—the same protocol used by `Ipopt`, `KNITRO`, and other AMPL-compatible solvers. The NL parser supports the full nonlinear expression DAG (directed acyclic graph) with automatic differentiation for gradient, Jacobian, and Hessian evaluation, eliminating the need for the user to supply derivative code.

External functions. Models that depend on user-supplied function libraries (for example IDAES property packages, custom thermodynamic correlations, or table-lookup routines) emit `f` opcodes in the NL stream that reference shared objects loaded at solve time. Starting in v0.7.1 `riopt` implements enough of AMPL’s `funcadd_AS` ABI to dlopen those libraries, register their imported functions, and dispatch `f` opcodes to them during forward and reverse sweeps of the autodiff tape. The same library handle is reused across every objective, gradient, Jacobian, and Hessian evaluation, and tangent / second-order requests are forwarded through the AMPL `arglist` structure, so external functions participate fully in second-order Newton steps. When a model declares external functions but no compatible library can be located, the parser surfaces a clear diagnostic rather than silently substituting zeros. This support is what enables Pyomo models that link against AMPL function libraries to be solved with `riopt` without modification.

This AMPL interface enables a Pyomo⁸ solver plugin via Pyomo’s ASL (AMPL Solver Library) infrastructure. The plugin registers `riopt` with Pyomo’s `SolverFactory`:

```
1 import pyomo_riopt # registers 'riopt' with SolverFactory
2 from pyomo.environ import *
3
4 model = ConcreteModel()
5 model.x = Var(initialize=1.5, bounds=(1, 5))
6 model.y = Var(initialize=1.5, bounds=(1, 5))
7 model.obj = Objective(expr=(model.x - 1)**2 + (model.y - 2)**2)
8 model.c1 = Constraint(expr=model.x + model.y <= 4)
9
10 solver = SolverFactory('riopt')
11 result = solver.solve(model, tee=True)
```

The plugin is a thin wrapper (30 lines of Python) that subclasses Pyomo’s ASL solver class. Pyomo handles model serialization to NL format and solution parsing from SOL format; the plugin only needs to locate the `riopt` executable and forward solver options. Installation is via `pip install pyomo-riopt`, which also registers the solver through a `pyomo.solvers` entry point for automatic discovery. The package is distributed as platform-specific binary wheels (Linux x86_64/aarch64, macOS x86_64/arm64, Windows amd64) with the compiled `riopt` binary bundled inside, so users get a working solver with just `pip install pyomo-riopt`—no Rust toolchain required. This means existing Pyomo workflows can switch from `Ipopt` to `riopt` by changing a single line (`SolverFactory('riopt')` instead of `SolverFactory('ipopt')`), with no changes to the model definition or solution retrieval code.

Solver options are passed through Pyomo’s standard `solver.options` dictionary:

```
1 solver = SolverFactory('riopt')
```

```

2 solver.options['tol'] = 1e-8
3 solver.options['max_iter'] = 1000
4 solver.options['print_level'] = 5

```

13.5 C API

riopt exposes a C API that closely mirrors the Ipopt C interface, enabling direct linking from C, C++, Python (via `ctypes` or `cffi`), Julia, and any language with C FFI support—without the subprocess and file I/O overhead of the NL/SOL interface.

13.5.1 Design

The API is surfaced by compiling `riopt` as a shared library (`cdylib` crate type, producing `libriopt.dylib` on macOS or `libriopt.so` on Linux). The library exports an opaque handle type and five function groups:

1. **Lifecycle:** `riopt_create` allocates a heap-owned problem handle from bounds, sizes, and callback pointers; `riopt_free` releases it.
2. **Options:** `riopt_add_num_option`, `riopt_add_int_option`, and `riopt_add_str_option` set solver parameters by Ipopt-compatible string keys (e.g. "tol", "max_iter", "mu_strategy").
3. **Solve:** `riopt_solve` takes the initial point in `x` and writes the primal solution back into `x`, with optional output pointers for constraint values, the objective, and all multipliers. Any output pointer may be `NULL`.

The callback signatures are identical to the Ipopt C interface:

```

1 typedef int (*Eval_F_CB)(int n, const double *x, int new_x,
2                          double *obj_value, void *user_data);
3 typedef int (*Eval_Grad_F_CB)(int n, const double *x, int new_x,
4                               double *grad_f, void *user_data);
5 typedef int (*Eval_G_CB)(int n, const double *x, int new_x,
6                           int m, double *g, void *user_data);
7 typedef int (*Eval_Jac_G_CB)(int n, const double *x, int new_x,
8                               int m, int nele_jac,
9                               int *iRow, int *jCol, double *values,
10                              void *user_data);
11 typedef int (*Eval_H_CB)(int n, const double *x, int new_x,
12                           double obj_factor, int m,
13                           const double *lambda, int new_lambda,
14                           int nele_hess,
15                           int *iRow, int *jCol, double *values,
16                           void *user_data);

```

Each callback returns 1 on success and 0 on failure. The Jacobian and Hessian callbacks implement the standard two-call protocol: when `=values == NULL=`, the callback fills `iRow/jCol` with the sparsity pattern (0-based indices); when `values != NULL=`, it fills numerical values in the same element order. The `new_x` and `new_lambda` flags are accepted and forwarded but currently ignored (`riopt` always re-evaluates), which is correct and slightly conservative.

13.5.2 Usage Example

The following skeleton shows the lifecycle for the HS071 problem (the full example is in `examples/c_api_test.c`):

```
1  #include "ripoint.h"
2
3  RipointProblem nlp = ripoint_create(
4      n, x_l, x_u, m, g_l, g_u,
5      nele_jac, nele_hess,
6      eval_f, eval_grad_f, eval_g, eval_jac_g, eval_h);
7
8  ripoint_add_int_option(nlp, "print_level", 5);
9  ripoint_add_num_option(nlp, "tol", 1e-8);
10
11 double x[4] = {1.0, 5.0, 5.0, 1.0}; /* initial point */
12 double obj_val;
13 int status = ripoint_solve(nlp, x, NULL, &obj_val,
14                           NULL, NULL, NULL, NULL);
15 /* x now holds the solution; status == 0 => Optimal */
16
17 ripoint_free(nlp);
```

Build and link from the repository root:

```
1 cargo build --release
2 cc examples/c_api_test.c -I. -Ltarget/release -lripoint \
3     -Wl,-rpath,$(pwd)/target/release -o c_api_test
4 ./c_api_test
```

On HS071 the C API test produces the correct solution ($f^* \approx 17.0140$, $x^* \approx [1.000, 4.743, 3.821, 1.379]$) in 11 IPM iterations.

13.5.3 Memory Safety and Thread Safety

All raw pointer dereferences in the C API implementation are confined to `unsafe` blocks. Each `CApiProblem` handle is heap-allocated and self-contained; the solver carries no global state, so `ripoint_solve` is re-entrant and multiple handles can be solved concurrently from different threads without synchronization.

13.6 GAMS Solver Link

To support GAMS (General Algebraic Modeling System),²⁰ `ripoint` includes a standalone solver link that bridges GAMS's GMO (GAMS Modeling Object) API to `ripoint`'s C API. The solver link is a shared library (`libGamsRipoint.dylib` on macOS, `libGamsRipoint.so` on Linux) that exports the standard GAMS solver entry points (`ripCreate`, `ripReadyAPI`, `ripCallSolver`, `ripFree`) and registers as an NLP solver in GAMS's solver configuration file.

The bridge implements five callbacks that delegate from `ripoint`'s Ipopt-compatible callback interface to GMO evaluation functions:

- **Objective and gradient:** `gmoEvalFuncObj` and `gmoEvalGradObj` evaluate the objective function and its dense gradient. For maximization problems, both are negated (`ripoint` always minimizes).

- **Constraints:** `gmoEvalFunc` evaluates each constraint row individually.
- **Jacobian:** The sparsity pattern is obtained once from `gmoGetMatrixRow` in CSR format and expanded to COO. Values are computed row-by-row via `gmoEvalGrad`, extracting sparse entries from the dense gradient at the structural nonzero positions.
- **Hessian:** `gmoHessLagStruct` provides the lower-triangular sparsity in COO format. `gmoHessLagValue` evaluates the weighted Hessian of the Lagrangian, with `=conweight = -1.0=` to account for the GAMS multiplier sign convention ($\pi_{\text{GAMS}} = -\lambda_{\text{Ipopt}}$).

Constraint bounds are derived from GAMS equation types (`==E==`, `==G==`, `==L==`, `==N==`) and the right-hand side. GAMS infinity values (`gmoPinf/gmoMinf`) are mapped to ripopt's $\pm 10^{30}$. When the analytical Hessian is not available (e.g., DNLP models), the solver link automatically enables L-BFGS approximation. GAMS iteration and resource limits are forwarded, and solver-specific options can be set via `ripoint.opt` files using the standard key-value format.

The solver link is built independently of the GAMS source tree (`make -C gams`) and installed by copying the library to the GAMS system directory and adding an entry to `gmscmpun.txt`. Once installed, GAMS models use ripopt with:

```
option nlp = ripopt;
Solve mymodel using nlp minimizing obj;
```

13.7 Julia/JuMP Interface

For Julia users, ripopt provides `Ripopt.jl`, a MathOptInterface (MOI)²¹ wrapper that integrates with JuMP.²² The wrapper enables ripopt to be used as a drop-in JuMP optimizer:

```
1 using JuMP, Ripopt
2
3 model = Model(Ripopt.Optimizer)
4 set_silent(model)
5
6 @variable(model, 1 <= x[1:4] <= 5)
7 set_start_value.(x, [1.0, 5.0, 5.0, 1.0])
8
9 @NLobjective(model, Min, x[1]*x[4]*(x[1]+x[2]+x[3]) + x[3])
10 @NLconstraint(model, x[1]*x[2]*x[3]*x[4] >= 25)
11 @NLconstraint(model, x[1]^2 + x[2]^2 + x[3]^2 + x[4]^2 == 40)
12
13 optimize!(model)
14 println(termination_status(model)) # LOCALLY_SOLVED
15 println(objective_value(model))    # 17.014
```

Solver options use the same string keys as Ipopt (`tol`, `max_iter`, `mu_strategy`, etc.) and are set via `set_optimizer_attribute`. Switching between ripopt and Ipopt requires only replacing `Ripopt.Optimizer` with `Ipopt.Optimizer`; the model definition is identical.

The implementation follows the incremental MOI interface: `Ripopt.Optimizer` implements `MOI.supports_incremental_interface`, `MOI.add_variable`, `MOI.add_constraint` (variable bounds), and `MOI.set` for `NLPBlock`, `ObjectiveSense`, `Silent`, `TimeLimitSec`, and pass-through `RawOptimizerAttribute`. Problem data (evaluator, sparsity structures,

objective sign) is passed to the five C callbacks through the `user_data` pointer as a GC-protected `Ref` to a `NamedTuple`; no closures are used, ensuring compatibility with Julia on ARM (AArch64), where closure-backed `@cfunction` trampolines are not supported. The library path is resolved at load time via Julia’s `__init__` hook so the precompiled image is not tied to a specific build directory.

14 Computational Results

We benchmark ripopt against Ipopt 3.14.19 (installed via Homebrew with MUMPS linear solver) on two test suites. All experiments are conducted on an Apple Mac Mini (aarch64-apple-darwin) with ripopt compiled in release mode (`-release`). Both solvers use default settings with a maximum of 3000 iterations.

14.1 Hock-Schittkowski Test Suite

The Hock-Schittkowski (HS) test suite²³ contains 120 nonlinear programming problems of varying size and difficulty, widely used for validating NLP solvers.

14.1.1 Solve Rates

Solver	Optimal	Failed	Total Solved
riopt	118	2	118/120 (98.3%)
Ipopt (native)	116	4	116/120 (96.7%)

riopt solves 118 of 120 problems, two more than Ipopt. ripopt exclusively solves 2 problems (HS214, HS223) that Ipopt fails on, while Ipopt does not exclusively solve any problem that ripopt cannot. ripopt’s 2 failures are `NumericalError` outcomes.

14.1.2 Solution Quality

On the 116 problems where both solvers converge:

- Matching objectives (relative difference $< 10^{-4}$): 110/116 (94.8%)

14.1.3 Speed

Metric	riopt	Ipopt (native)
Median per-problem time	83 μ s	1.9 ms
Total time (116 problems)	46.4 ms	285.7 ms
Geometric mean speedup	20.8 $\times$	—

riopt is faster on 114/116 commonly-solved problems (98%), with 103 problems showing $10\times+$ speedup and a median speedup of $20.9\times$. The per-iteration speedup is attributable to stack allocation, the absence of C/Fortran interop, cache-efficient dense linear algebra, and Mehrotra predictor-corrector with Gondzio centrality corrections (enabled by default).

14.1.4 Iteration Counts

Statistic	riopt	Ipopt (native)
Mean	67.0	13.3
Median	13	10

Ipopt uses fewer iterations on average, reflecting its more mature barrier parameter and line search tuning. ripoft’s lower per-iteration cost more than compensates, yielding the observed 20.8× wall-clock speedup. The mean iteration count is driven by a small tail of problems (notably parameter-fitting problems and the two ripoft-only solves) that require many cheap iterations to reach strict convergence; the median iteration count is unchanged.

14.2 CUTEst Benchmark Suite

The CUTEst suite⁷ provides 727 nonlinear programming problems from diverse application domains, representing the standard benchmark for NLP solver evaluation.

14.2.1 Overall Solve Rates

Metric	riopt	Ipopt
Total solved	551/727 (75.8%)	556/727 (76.5%)
Both solve	529	529
riopt only	22	—
Ipopt only	—	27

All counts in this section are strict **Optimal** (Ipopt status 0 or ripoft **SolveStatus::Optimal**); **Acceptable**, **MaxIterations**, **MaxTimeExceeded**, and **NumericalError** outcomes are **not** counted as solved, per the project honesty rule (see **CLAUDE.md**). At v0.8.1, Ipopt edges ripoft on CUTEst by 5 strict-Optimal problems (556 vs 551). ripoft exclusively solves 22 problems that Ipopt cannot; Ipopt exclusively solves 27 that ripoft cannot. Of ripoft’s 22 exclusive wins, 12 are nonlinear-equation (NE) systems where the CUTEst Ipopt wrapper rejects with **IpoptStatus(-10)** ("insufficient degrees of freedom") and ripoft’s implicit-slack IPM accepts them as standard equality-constrained NLPs; the remaining 10 are **MaxIterations** or **RestorationFailed** cases recovered by ripoft’s restoration cascade or fallback solvers. The v0.7→v0.8 alignment effort traded a handful of benchmark-tuned heuristics for Ipopt-faithful kernels (filter $\leq$ tolerance, **AugmentFilter** envelope, post-restoration handoff, **LoqoMuOracle::CalculateM** soft-restoration E_μ , watchdog/ $\mu_{\max}$ capture); the net effect on this suite, measured against strict-Optimal counts only, is an aggregate change of -5 on the 727-problem corpus, with the surviving wins now attributable to the implicit-slack KKT formulation and the restoration architecture rather than to retired heuristics.

14.2.2 Solution Quality

On the 529 problems where both solvers strictly converge:

- Matching objectives (relative difference $< 10^{-4}$): 514/529 (97.2%)

- 15 mismatches: both reach valid KKT points but at different local optima (including cases where fallback solvers find different basins)

14.2.3 Speed Comparison

On the 529 commonly-Optimal problems:

Metric	Value
Geometric mean speedup (riopt faster)	7.9×
Median speedup (riopt faster)	10.6×
Problems where ript is faster	90%
Problems where ript is 10×+ faster	54%

The v0.8 default linear-solver backend is the pure-Rust **feral** multifrontal LDL^T (Section [Linear Algebra](#)), developed in a separate repository with its own benchmark corpus; **rmumps** remains available behind the opt-in **rmumps** feature for A/B comparison and as a regression-hunting baseline. Both backends are pure Rust — v0.7 also had no FFI step — so the v0.8 change is about debuggability and component isolation rather than build simplification. The speedup gap between v0.7 and v0.8 narrowed slightly because **feral** has not yet matched Fortran MUMPS’s GEMM throughput on the largest sparse problems, but the same per-iteration overhead advantage on small/medium problems persists.

Mean and median iteration counts on the commonly-solved set are within a few iterations between the two solvers (43.4 vs 35.5 mean, 13 vs 12 median), so the wall-clock advantage is per-iteration overhead, not algorithmic shortcut.

14.2.4 Attribution of ript-only Solves

The 22 CUTEst problems solved by ript to strict **Optimal** but not by Ipopt at v0.8.1 break down as:

- **Implicit-slack acceptance of NE systems** (12 problems): Nonlinear equation systems where the CUTEst Ipopt wrapper rejects the model upfront with **IpoptStatus(-10)** ("insufficient degrees of freedom" — the wrapper requires a stationary objective). ript’s implicit-slack IPM treats these as ordinary equality-constrained NLPs with a zero objective: the convergence test reduces to feasibility, the KKT system is well-posed, and the standard barrier/restoration machinery suffices. No problem-specific reformulation, residual-stuffing, or Gauss-Newton-on-the-residuals path is involved (the dedicated NE-to-LS reformulation that v0.7 used was removed in v0.8; the wins survived the removal). Representative examples include **BEALENE**, **BIGGS6NE**, **BOX3NE**, **DENSCHNBNE**, **DEVGLA1NE**, **DEVGLA2NE**, **ENGVAL2NE**, **EXP2NE**, **HS25NE**, **LANCZOS1**, **LEVYMON5**, **NYSTROM5**.
- **Two-phase restoration and fallback solvers** (~7 problems): Problems where Ipopt fails with **MaxIterations** or **RestorationFailed** but ript’s Gauss-Newton restoration phase, multi-attempt recovery (barrier-perturbation cascade), or one of the fallback solvers (L-BFGS, augmented Lagrangian, SQP, explicit slack) drives the iterate to a strict-tolerance KKT point.

- **Other algorithmic differences** (~ 3 problems): Differences in early-termination handling, status mapping, or scaling that allow ripopt to converge where Ipopt’s CUTEst interface returns a non-Optimal status.

Conversely, Ipopt exclusively solves 27 problems at v0.8.1 that ripopt cannot. These are primarily nonconvex problems where Ipopt’s mature inertia handling and barrier-parameter strategy give it an advantage. ripopt’s dominant remaining failure modes on this suite at v0.8.1 are `RestorationFailed`, `LocalInfeasibility`, `MaxTimeExceeded`, `MaxIterations`, and `NumericalError`. The shift in profile relative to v0.7.1 (where `NumericalError` dominated at 83 cases) reflects the Ipopt-aligned post-restoration handoff and stricter convergence test now correctly classifying iterates as restoration-stalled or time-bound rather than as numerical breakdown—honest signal where the solver is actually getting stuck.

14.3 Domain-Specific Benchmarks

To complement the general-purpose benchmark suites, we evaluate ripopt on five domain-specific problem sets. The first three (electrolyte speciation, AC optimal power flow, and parameter estimation) are formulated in Pyomo and solved via the AMPL NL interface. The remaining two (gas pipeline networks and water distribution networks) are taken directly as AMPL `.nl` files from external benchmark libraries.

14.3.1 Electrolyte Speciation

A set of 13 seawater electrolyte speciation problems (computing equilibrium concentrations of dissolved species) provides a challenging test of nonlinear equation solving. ripopt solves all 13 problems (100%), while Ipopt solves 12 of 13 (92.3%)—failing on the full seawater speciation problem. These are square nonlinear equation systems with a dummy zero objective; ripopt’s implicit-slack IPM accepts them as ordinary equality-constrained NLPs and drives the residual to tolerance through the standard barrier/restoration path.

14.3.2 AC Optimal Power Flow (Electrical Grid)

Four AC optimal power flow problems on small IEEE/PGLib test cases (`case3_lmld`, `case5_pjm`, `case14_ieee`, `case30_ieee`) test performance on structured engineering optimization. Both ripopt and Ipopt solve all 4 problems to strict `Optimal` at v0.8.1; `case30_ieee`, which had previously exhausted the iteration budget at v0.7.1 under the stricter KKT backward-error probe, now converges under the v0.8 post-restoration handoff and lands at the same local optimum as Ipopt ($\$8208.52/h$). On the four commonly-solved cases ripopt’s median speedup over Ipopt is $2.1\times$ (geometric mean $1.5\times$); the gap is smaller than on the small-problem CUTEst tail because dense Bunch-Kaufman is no longer the bottleneck and feral has not yet matched MUMPS’s GEMM throughput on the largest sparse fronts. The suite is named `grid` in the repository to make its purpose obvious in the directory layout.

14.3.3 Parameter Estimation

A large-scale parameter estimation problem (CHO cell growth kinetics, $n = 21,672$ variables, $m = 21,660$ constraints) tests scalability. Neither solver converges within the iteration limit (both return `MaxIterations`), indicating that this problem class requires specialized techniques beyond standard interior point methods.

14.3.4 Gas Pipeline Networks

Four gas pipeline NLPs taken from the `Sakshi21299/gas_networks` repository (steady-state and 24-hour dynamic operating conditions on the GasLib-11 and GasLib-40 networks, n ranging from 204 to 21,058) test ripopt on equality-only finite-volume PDE discretizations of the Euler equations. These problems were the original motivation for the sparse Gauss-Newton restoration phase (Section [Two-Phase Restoration with Multi-Attempt Recovery](#)): on `gaslib11_steady` the dense Bunch-Kaufman bottleneck took ~ 6 s per restoration step before the sparse refactor and ~ 0.02 s afterward. The Logo μ oracle (enabled by default in 0.6.2) reduces this problem from 164 iterations with one NLP restoration to 134 iterations with zero. The suite is solved problem-by-problem via the AMPL `.nl` interface and does not feed the composite report.

14.3.5 Water Distribution Networks

Six water distribution network design instances from MINLPLib²⁴ exercise ripopt on signomial Hazen-Williams head-loss formulations with mass-balance and pressure constraints. Five of the six are mixed-binary NLPs (MBNLP) treated as continuous relaxations by both solvers. On the pure-NLP `water` instance, ripopt converges to the best-known primal bound of 963.13 reported by MINLPLib, while Ipopt reaches a different stationary point at 1001.16—a useful demonstration that the multi-solver fallback architecture (two-phase restoration, explicit slack) can find better local minima on non-convex problems where Ipopt’s default barrier path commits to a worse basin. Like the gas suite, this set is solved via the AMPL `.nl` interface and does not feed the composite report.

14.4 Failure Analysis

14.4.1 Common Failure Patterns

At v0.8.1, the 176 ripopt failures on CUTEst (727 problems, 551 strict `Optimal`) break down as:

- **RestorationFailed**: 70 cases (39.8%). The dominant failure mode. Both GN and NLP restoration cycles fail to reduce constraint violation enough to escape the basin; this category subsumed many cases that v0.7 misclassified as `NumericalError` once the post-restoration handoff was aligned with Ipopt 3.14.
- **LocalInfeasibility**: 29 cases (16.5%). The solver reaches a stationary point of the constraint violation function and declares local infeasibility.

- **Acceptable:** 20 cases (11.4%). Solver hit the `acceptable` relaxed tolerance but not strict `Optimal`. Per the project honesty rule (`CLAUDE.md`) these are not counted as solved.
- **MaxTimeExceeded:** 19 cases (10.8%). Wall-clock budget exhausted before strict tolerance was reached. The `MaxTimeExceeded` variant was added in v0.8.1 to distinguish wall-clock cutoffs from iteration-count cutoffs.
- **MaxIterations:** 18 cases (10.2%). Solver makes progress but does not reach strict tolerance within the 2999-iteration budget.
- **NumericalError:** 9 cases (5.1%). Catastrophic step-computation failures (factor-cache breakdown, NaN propagation) not recoverable by inertia correction.
- **Timeout / StopAtTinyStep / EvaluationError:** 11 cases combined (6.3%). Solver-side wall-clock timeout, α -tiny stalls, or function-evaluation NaNs.

14.4.2 Comparison of Failure Modes

Failure Mode	riopt	Ipopt
RestorationFailed	70	4
LocalInfeasibility / Infeasible	29	11
Acceptable (relaxed tol only)	20	5
MaxTimeExceeded	19	0
MaxIterations	18	12
Timeout	6	10
NumericalError	9	0
EvaluationError	3	0
StopAtTinyStep	2	0
IpoptStatus(-10)	—	123
ErrorInStepComputation	0	2
InvalidNumberDetected	0	1
Other (status 3, 4, etc.)	0	3
Total failures	176	171

The failure profiles reveal fundamentally different bottlenecks. Ipopt’s dominant failure mode is `IpoptStatus(-10)` (123 cases)—the CUTEst Ipopt wrapper rejecting problems with insufficient degrees of freedom (notably nonlinear equation systems posed with a flat objective). rlopt’s implicit-slack IPM accepts these as ordinary equality-constrained NLPs, which is the source of most of its 22 exclusive wins.

riopt’s dominant failure mode is `RestorationFailed` (70 cases, 39.8%), followed by `LocalInfeasibility` (29 cases, 16.5%) and `Acceptable` (20 cases, 11.4%). Per the project honesty rule (`CLAUDE.md`) `Acceptable` outcomes are counted as failures here, not as solves. The 176 rlopt failures compare to Ipopt’s 171 — a 5-failure gap in Ipopt’s favor on strict convergence, narrowed (and reversed) by rlopt’s speed advantage on the commonly-Optimal set.

14.5 Speed vs. Accuracy Trade-off

The benchmarks reveal a clear trade-off:

- **Speed:** ripopt is dramatically faster per iteration on small problems and moderately faster on medium problems, with a CUTEst geometric-mean wall-clock speedup of $7.9\times$ (median $10.6\times$) at v0.8.1 on the 529 commonly-Optimal problems. This advantage comes from implementation characteristics (Rust, stack allocation, no C/Fortran interop into the IPM kernel) rather than algorithmic differences; mean and median iteration counts are within a few iterations of Ipopt’s (43.4 vs 35.5 mean, 13 vs 12 median on the commonly-solved CUTEst set).
- **Iteration count:** With the v0.8 alignment of the Loqo and Quality-Function oracles to Ipopt’s faithful implementations, the residual iteration-count gap is concentrated on a small number of problems where filter and inertia-correction interactions still differ (Section [Opportunities for Improvement](#)).
- **Robustness:** ripopt’s two-phase restoration, multi-attempt recovery, and multi-solver fallback architecture provide complementary recovery mechanisms. At v0.8.1 ripopt exclusively solves 22 CUTEst problems that Ipopt cannot, while Ipopt exclusively solves 27 that ripopt cannot. With 176 CUTEst failures vs. Ipopt’s 171, the two solvers are roughly tied on aggregate failures, though the profiles differ qualitatively: ripopt’s failures are dominated by `RestorationFailed` and `LocalInfeasibility`, while Ipopt’s are dominated by `IpoptStatus(-10)` rejections on nonlinear equation systems that ripopt’s implicit-slack IPM accepts as ordinary equality-constrained NLPs.
- **Solution quality:** On the 529 commonly-Optimal CUTEst problems, the objective match rate at the 10^{-4} relative tolerance is 97.2% (514/529). The remaining 2.8% are different local optima from algorithmic path differences (including fallback solvers finding different basins), not accuracy deficiencies.

15 Parametric Sensitivity Analysis

ripopt provides sIPOPT-style²⁵ post-optimal sensitivity analysis. Given a converged solution (x^*, y^*, z_L^*, z_U^*) , the sensitivity of the optimal point to parameter perturbations Δp is computed by a single backsolve of the KKT system:

$$\begin{bmatrix} W + \Sigma & J^T \\ J & 0 \end{bmatrix} \begin{bmatrix} dx \\ dy \end{bmatrix} = - \begin{bmatrix} \nabla_{xp}^2 L \cdot \Delta p \\ \frac{\partial g}{\partial p} \cdot \Delta p \end{bmatrix} \quad (21)$$

where W is the Hessian of the Lagrangian and $\Sigma = Z_L S_L^{-1} + Z_U S_U^{-1}$ is the barrier diagonal at the solution. Bound multiplier sensitivities are recovered from the complementarity conditions: $dz_{L,i} = -z_{L,i} dx_i / (x_i - x_{L,i})$ and $dz_{U,i} = z_{U,i} dx_i / (x_{U,i} - x_i)$.

15.1 Implementation

The implementation uses the simpler post-solve approach: `solve_with_sensitivity()` calls the standard `solve()`, then reassembles the *unregularized* KKT matrix at the solution point and factors it. This avoids modifying the core IPM loop entirely. The cost is one extra Hessian/Jacobian evaluation and one factorization—negligible compared to the solve itself.

The reassembled KKT omits the inertia correction terms (δ_w , δ_c) that the final IPM iteration may have applied. This is important: the regularized matrix would introduce $O(\delta)$ bias in the sensitivities, while the unregularized matrix gives the exact first-order sensitivity.

Users implement the `ParametricNlpProblem` trait, which extends `NlpProblem` with five methods specifying the number of parameters and the sparsity/values of $\partial g/\partial p$ and $\nabla_{xp}^2 L$. The `SensitivityContext` retains the factored KKT and supports:

- `compute_sensitivity()`: computes dx/dp , dy/dp , dz_L/dp , dz_U/dp for one or more perturbation vectors Δp
- `reduced_hessian()`: extracts the $n \times n$ upper-left block of M^{-1} for covariance estimation in parameter estimation problems

15.2 Example: HS071 with Parametric Constraint Bound

Consider the HS071 problem where the equality constraint bound $p = 40$ is treated as a parameter. After solving at $p = 40$, sensitivity analysis predicts the solution at $p = 40.1$ by computing $x(p + \Delta p) \approx x^* + (dx/dp) \cdot \Delta p$:

Variable	$x^*(p=40)$	Predicted $x(p=40.1)$	Actual $x(p=40.1)$	Error
x_1	1.000000	1.000000	1.000000	$< 10^{-8}$
x_2	4.743000	4.751642	4.751634	8.4×10^{-6}
x_3	3.821150	3.824904	3.824896	8.0×10^{-6}
x_4	1.379408	1.375540	1.375553	1.4×10^{-5}

The prediction errors are $O(\Delta p^2)$ as expected from first-order Taylor expansion, confirming correctness of the sensitivity computation.

16 Opportunities for Improvement

The failure analysis and benchmark comparison reveal several concrete opportunities for improvement. At v0.8.1 the CUTEst failure profile is dominated by `RestorationFailed` (70 cases, 44.9%), followed by `LocalInfeasibility` (29, 18.6%), `MaxTimeExceeded` (19, 12.2%), `MaxIterations` (18, 11.5%), `NumericalError` (9, 5.8%), and a tail of `Timeout` / `EvaluationError` / `StopAtTinyStep` (11 combined). The shift from v0.7.1 (where `NumericalError` dominated at 76 cases) reflects the v0.8 Ipopt-aligned post-restoration handoff and stricter convergence test correctly classifying iterates as restoration-stalled or time-bound rather than as numerical breakdown. This profile suggests priorities for future work.

16.1 Restoration-Phase Robustness

`RestorationFailed` is the largest category at v0.8.1 (70 cases). The Ipopt-aligned handoff (Section [Post-Restoration State Handoff](#)) eliminated the post-handoff μ blow-up that had inflated the v0.7 `NumericalError` bucket, but did not fix the underlying issue that on a sizeable subset of problems neither the GN feasibility solver nor the multi-attempt μ cycle escapes the basin. Promising directions include: (i) coupling the GN regularization ϵ to the local Jacobian conditioning so the solver does not stall in a Levenberg-Marquardt damping plateau; (ii) replacing the deterministic x -perturbation in the recovery cascade with a low-discrepancy or basin-hopping schedule; and (iii) using a feasibility-restoration phase that admits constraint scaling updates between attempts.

16.2 Infeasibility Detection and Recovery

The `LocalInfeasibility` failure mode (29 cases at v0.8.1) occurs when the solver reaches a stationary point of the constraint violation function $\|\nabla_x \theta(x)\| \approx 0$. While many of these are genuinely infeasible, some may represent false declarations where a feasible solution exists in a different basin. Several strategies could reduce false positives:

- **Multi-start for NE problems:** When the IPM converges to a stationary point of $\|g(x) - t\|$ that is nonzero on a square NE system ($m = n$), a second attempt from a perturbed starting point could escape the basin.
- **Second-order infeasibility analysis:** Using the Hessian of $\theta(x)$ at the stationary point to distinguish between local minima (genuinely infeasible) and saddle points (potentially feasible nearby) could reduce false declarations.
- **Constraint-space search:** Projecting the iterate onto the constraint manifold via a sequence of Newton steps on individual constraint subsets, rather than minimizing total violation, could find feasible points that the global violation minimization misses.

16.3 Adaptive Barrier Parameter Tuning

With the v0.8 oracles already faithful ports of Ipopt’s `LoqoMuOracle` and `QualityFunctionMuOracle`, the residual gap on iteration counts is dominated by interactions between the oracle output and the post-restoration handoff (where the parent inherits the pre-restoration μ rather than recomputing one). A more careful warm-up policy after restoration—e.g., a single LOQO call gated on a feasibility check, or a brief quality-function evaluation against the recovered iterate—could reduce iteration counts on problems that currently need 50–100 outer iterations to push μ back down after a successful restoration handoff.

16.4 Multiplier Initialization

riopt initializes constraint multipliers y to zero and relies on the KKT system to determine appropriate values. Ipopt’s least-squares multiplier initialization computes an initial y from the normal equations $J^T y \approx -\nabla f$, which can significantly reduce iteration counts, especially

when the initial point is near-feasible. Adding this initialization to ripopt is straightforward and could improve convergence on problems where the initial $y = 0$ leads to poor early search directions.

16.5 Performance Optimization for Large Problems

The 6 `Timeout`, 19 `MaxTimeExceeded`, and 18 `MaxIterations` failures indicate performance and convergence-rate bottlenecks on larger problems. Two directions could help:

- **Sparse Jacobian assembly:** For over-constrained problems (e.g., EXPFITC with $m = 502$, $n = 5$), the condensed KKT reduces the system to dimension n but the dense Jacobian assembly ($O(mn)$) still dominates. Exploiting Jacobian sparsity in the condensation step could reduce per-iteration cost.
- **Sparse factorization throughput on the largest fronts:** At v0.8.1 the default `feral` multifrontal backend matches Ipopt’s MUMPS-backed factorization closely on small and medium problems, but trails on the largest sparse fronts where MUMPS leverages decades of GEMM-throughput tuning. Improvements in `feral`’s BLAS-3 supernodal kernels, parallelization across independent fronts, and a true incomplete- $LDL^T + \text{MINRES}$ hybrid (deferred from v0.8 to v0.9) would close the remaining gap. The legacy `rmumps` backend remains available for benchmarks that require a direct A/B against Fortran MUMPS.

16.6 Quasi-Newton as Primary Solver

The L-BFGS solver (Section [L-BFGS for Unconstrained Problems](#)) is currently used as a first-attempt solver for unconstrained problems and a fallback for failed IPM solves. Extending it to serve as a primary solver for problems without second-order derivative information—or integrating L-BFGS curvature updates within the IPM framework as a Hessian approximation—could further broaden ripopt’s applicability.

16.7 Scaling

ripopt implements gradient-based NLP scaling, which normalizes the objective gradient and constraint Jacobian rows to have maximum absolute value of 1.0 at the initial point, and exposes a user-supplied variable-scaling hook through `user_x_scaling` (Section [NLP Scaling](#)). More sophisticated automatic scaling strategies—iterative equilibration on the full KKT matrix, second-order-aware scaling, or interleaving Ruiz-style symmetric equilibration with gradient-based row scaling—could further improve performance on problems with extreme scaling imbalances where the current heuristic chooses a poor reference point.

16.8 Summary of Estimated Impact

Improvement Area	Current Failures Addressed	Estimated Problems Recovered
Restoration-phase robustness	70 RestorationFailed	15–30
Convergence rate / max-iter	18 MaxIterations + 19 MaxTimeExc	8–15
Infeasibility detection	29 LocalInfeasibility	3–6
Numerical-error recovery	9 NumericalError	2–4
Multiplier initialization	Various	2–4
Scaling	Various	2–5

The largest opportunity at v0.8.1 is recovering problems that now fail as **RestorationFailed**. The Ipopt-aligned post-restoration handoff fixed the post-restoration μ blow-up that had previously masqueraded as **NumericalError**, but did not address the underlying restoration-cycling problem on a sizeable subset of inputs. Together, these improvements are largely independent and could collectively push ripopt’s CUTEst solve rate toward 82–87%.

17 Related Work

Interior point methods for nonlinear programming have a rich history. The barrier approach originates with,² and the primal-dual formulation was developed for linear programming by²⁶ and extended to nonlinear programming by²⁷ and.⁴ The filter line search was introduced by⁵ and adapted for interior point methods by.⁴ The adaptive barrier parameter strategy used in Ipopt was formalized by.⁶

The trust region framework²⁸ provides an alternative globalization strategy. Condensed KKT formulations (Schur complement reductions) are well-known in the optimization literature¹¹ but are typically presented as optional reductions of the explicit-slack system; our use of implicit slacks as the primary formulation is less common.

Uno (Unifying Nonlinear Optimization)²⁹ is a recent C++ framework that decomposes Lagrange-Newton methods (SQP and interior point) into interchangeable building blocks—constraint relaxation strategies, subproblem solvers, and globalization mechanisms—that can be combined via configuration. Uno can mimic existing solvers (including Ipopt) through presets while also enabling novel combinations. In contrast, ripopt takes a fixed algorithmic approach (primal-dual barrier with filter line search) but innovates within that framework through implicit slacks, two-phase restoration, and automatic problem reformulation.

The Bunch-Kaufman factorization¹⁵ is standard for symmetric indefinite systems. The Gauss-Newton method and Levenberg-Marquardt regularization^{12,13} are standard tools for nonlinear least-squares; our contribution is their application within the restoration phase of an interior point solver. Parametric sensitivity analysis for NLPs was formalized in the SIPOPT framework,²⁵ which reuses the factored KKT matrix from the final IPM iteration to compute solution sensitivities with respect to problem parameters at near-zero cost.

The use of iterative methods for KKT systems in interior point solvers has been explored by several authors. The MINRES algorithm⁹ is the natural choice for symmetric indefinite systems, as it minimizes the residual norm with short recurrences. Incomplete factorization preconditioners^{10,18} are widely used to accelerate Krylov methods for sparse linear systems; our contribution is the integration of an incomplete multifrontal LDL^T fac-

torization (sharing the same elimination tree as the direct solver) as a preconditioner within a hybrid direct-iterative framework that automatically adapts to problem difficulty. Nested dissection ordering¹⁷ with multilevel refinement¹⁶ is standard for fill-reducing orderings of sparse matrices from finite element discretizations.

18 Conclusions

We have presented ripopt, an interior point optimizer in approximately 21,700 lines of Rust that achieves competitive performance with Ipopt through a combination of architectural simplification, algorithmic innovation, and a multi-solver fallback architecture. The implicit slack formulation produces smaller KKT systems and a cleaner interface. The Ipopt-aligned two-gate convergence test, two-phase restoration, preprocessing, near-linear constraint detection, and fallback solvers (L-BFGS, augmented Lagrangian, SQP, explicit slack) compensate for the challenges introduced by eliminating explicit slack variables and provide novel algorithmic capabilities. A configurable linear solver infrastructure—offering direct multifrontal, iterative (preconditioned MINRES), and hybrid direct-iterative modes—enables ripopt to scale to larger problems while maintaining robustness through automatic mode switching.

On the Hock-Schittkowski test suite (historical, retired from the v0.8 harness), ripopt solved 118 of 120 problems (98.3%), surpassing Ipopt’s 116 (96.7%). On the CUTEst benchmark suite at v0.8.1, ripopt strictly solves 551 of 727 problems (75.8%) versus Ipopt’s 556 (76.5%)—a 5-problem gap in Ipopt’s favor on strict convergence, narrowed (and reversed) by ripopt’s 7.9× geometric-mean wall-clock speedup (median 10.6×) on the 529 commonly-Optimal problems. ripopt exclusively solves 22 problems Ipopt cannot, and Ipopt exclusively solves 27 that ripopt cannot. The speed advantage makes ripopt particularly attractive for applications requiring rapid solves of small-to-medium NLPs, such as model predictive control, real-time optimization, and embedded systems. The v0.8 release line moves the default sparse linear solver from `rmumps` (an embedded pure-Rust port that lived inside the ripopt tree) to `feral`,¹ an independently developed and independently benchmarked pure-Rust multifrontal LDL^T solver. The build remains entirely Rust; the gain is debuggability, since linear-solver failures can now be reproduced and isolated against feral’s own test corpus rather than being entangled with ripopt’s IPM-driven workload. The cost is a modest GEMM-throughput gap on the largest sparse fronts.

The failure analysis reveals that `RestorationFailed` is now the dominant failure mode on CUTEst (70 cases, 39.8% of failures), followed by `LocalInfeasibility` (29, 16.5%), `Acceptable` (20, 11.4%; counted as a failure per the project honesty rule), `MaxTimeExceeded` (19, 10.8%), and `MaxIterations` (18, 10.2%). The shift from v0.7.1’s `NumericalError`-dominated profile reflects the v0.8 Ipopt-aligned post-restoration handoff correctly classifying iterates as restoration-stalled or time-bound rather than as numerical breakdown—an honest signal where the solver is actually getting stuck. Targeting restoration-phase robustness, multiplier initialization, and large-problem factorization throughput could push the CUTEst strict-Optimal rate above 80\%.

The built-in AMPL NL interface and Pyomo solver plugin enable ripopt to serve as a drop-in replacement for Ipopt in existing Pyomo workflows, requiring only a one-line change to switch solvers. The GAMS solver link extends this accessibility to the GAMS modeling

ecosystem, enabling `=option nlp = ripopt;` with full Hessian support and option file integration. The sIPOPT-style parametric sensitivity analysis enables post-optimal what-if analysis and uncertainty propagation at near-zero cost, extending ripopt’s utility beyond single-solve optimization.

The source code for ripopt is available at <https://github.com/jkitchin/riport> under the EPL-2.0 license.

19 References

References

- [1] John R. Kitchin. feral: a pure-Rust multifrontal sparse symmetric indefinite LDL^T solver. Software, version 0.1.0, 2026. URL <https://crates.io/crates/feral>.
- [2] Anthony V Fiacco and Garth P McCormick. *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*. John Wiley & Sons, 1968. URL <https://doi.org/10.1137/1.9781611971316>. Reprinted as SIAM Classics in Applied Mathematics 4, 1990.
- [3] Anders Forsgren, Philip E Gill, and Margaret H Wright. Interior methods for nonlinear optimization. *SIAM Review*, 44(4):525–597, 2002. URL <https://doi.org/10.1137/S0036144502414942>.
- [4] Andreas Wächter and Lorenz T Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006. URL <https://doi.org/10.1007/s10107-004-0559-y>.
- [5] Roger Fletcher and Sven Leyffer. Nonlinear programming without a penalty function. *Mathematical Programming*, 91(2):239–269, 2002. URL <https://doi.org/10.1007/s101070100244>.
- [6] Jorge Nocedal, Andreas Wächter, and Richard A Waltz. Adaptive barrier update strategies for nonlinear interior methods. *SIAM Journal on Optimization*, 19(4):1674–1693, 2008. URL <https://doi.org/10.1137/060649513>.
- [7] Nicholas IM Gould, Dominique Orban, and Philippe L Toint. CUTEst: a constrained and unconstrained testing environment with safe threads for mathematical optimization. *Computational Optimization and Applications*, 60:545–557, 2015. URL <https://doi.org/10.1007/s10589-014-9687-3>.
- [8] Michael L Bynum, Gabriel A Hackebeil, William E Hart, Carl D Laird, Bethany L Nicholson, John D Sirola, Jean-Paul Watson, and David L Woodruff. *Pyomo—Optimization Modeling in Python*, volume 67 of *Springer Optimization and Its Applications*. Springer, 3rd edition, 2021. URL <https://doi.org/10.1007/978-3-030-68928-5>.

- [9] Christopher C Paige and Michael A Saunders. Solution of sparse indefinite systems of linear equations. *SIAM Journal on Numerical Analysis*, 12(4):617–629, 1975. URL <https://doi.org/10.1137/0712047>.
- [10] J A Meijerink and H A van der Vorst. An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix. *Mathematics of Computation*, 31(137):148–162, 1977. URL <https://doi.org/10.1090/S0025-5718-1977-0438681-4>.
- [11] Jorge Nocedal and Stephen J Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, 2nd edition, 2006. URL <https://doi.org/10.1007/978-0-387-40065-5>.
- [12] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944. URL <https://doi.org/10.1090/qam/10666>.
- [13] Donald W Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963. URL <https://doi.org/10.1137/0111030>.
- [14] Robert J Vanderbei. LOQO: an interior point code for quadratic programming. *Optimization Methods and Software*, 11(1-4):451–484, 1999. URL <https://doi.org/10.1080/10556789908805759>.
- [15] James R Bunch and Linda Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Mathematics of Computation*, 31(137):163–179, 1977. URL <https://doi.org/10.1090/S0025-5718-1977-0428694-0>.
- [16] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. URL <https://doi.org/10.1137/S1064827595287997>.
- [17] J Alan George. Nested dissection of a regular finite element mesh. *SIAM Journal on Numerical Analysis*, 10(2):345–363, 1973. URL <https://doi.org/10.1137/0710032>.
- [18] Edmond Chow and Yousef Saad. Experimental study of ILU preconditioners for indefinite matrices. *Journal of Computational and Applied Mathematics*, 86(2):387–414, 1997. URL [https://doi.org/10.1016/S0377-0427\(97\)00171-2](https://doi.org/10.1016/S0377-0427(97)00171-2).
- [19] David M Gay. Writing .nl files. Technical Report SAND2003-3492, Sandia National Laboratories, 2003. Updated 2005. Available at <https://ampl.com/resources/booking-your-solver-to-ampl/>.
- [20] Michael R Bussieck and Alex Meeraus. The GAMS system. In Josef Kallrath, editor, *Modeling Languages in Mathematical Optimization*, volume 88 of *Applied Optimization*, pages 137–157. Springer, 2004. URL https://doi.org/10.1007/978-1-4613-0215-5_8.

- [21] Benoît Legat, Oscar Dowson, Joaquim Dias Garcia, and Miles Lubin. MathOptInterface: a data structure for mathematical optimization problems. *INFORMS Journal on Computing*, 34(2):672–689, 2022. URL <https://doi.org/10.1287/ijoc.2021.1083>.
- [22] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 15:581–589, 2023. URL <https://doi.org/10.1007/s12532-023-00239-3>.
- [23] Willi Hock and Klaus Schittkowski. *Test Examples for Nonlinear Programming Codes*, volume 187 of *Lecture Notes in Economics and Mathematical Systems*. Springer, 1981. URL <https://doi.org/10.1007/978-3-642-48320-2>.
- [24] Michael R. Bussieck, Arne S. Drud, and Alexander Meeraus. MINLPLib — a collection of test models for mixed-integer nonlinear programming. *INFORMS Journal on Computing*, 15(1):114–119, 2003. URL <https://doi.org/10.1287/ijoc.15.1.114.15159>.
- [25] Hans Pirnay, Rodrigo López-Negrete, and Lorenz T Biegler. Optimal sensitivity based on IPOPT. *Mathematical Programming Computation*, 4(4):307–331, 2012. URL <https://doi.org/10.1007/s12532-012-0043-2>.
- [26] Sanjay Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2(4):575–601, 1992. URL <https://doi.org/10.1137/0802028>.
- [27] Richard H Byrd, Mary E Hribar, and Jorge Nocedal. An interior point algorithm for large-scale nonlinear programming. *SIAM Journal on Optimization*, 9(4):877–900, 1999. URL <https://doi.org/10.1137/S1052623497325107>.
- [28] Andrew R Conn, Nicholas IM Gould, and Philippe L Toint. *Trust Region Methods*. MPS/SIAM Series on Optimization. SIAM, 2000. URL <https://doi.org/10.1137/1.9780898719857>.
- [29] Charlie Vanaret and Sven Leyffer. Implementing a unified solver for nonlinearly constrained optimization. Submitted to Mathematical Programming Computation. Preprint available at <https://www.researchgate.net/publication/397446552>, 2024.